Claude Orchestration Framework

A reusable multi-agent orchestration setup for Claude Code

Generated 2026-05-02 (rev 4 — pre-flight safety)

Cover

This document is the printable version of the **Claude Orchestration Framework** — a tech-stack-agnostic multi-agent setup distilled from a real production codebase.

The framework lives at ~/Desktop/claude-orchestration-framework/ (local) and on GitHub at https://github.com/abhinavsehgal/claude-orchestration-framework.

What's inside (in this order):

1. **README** — purpose + quick-start for new and existing projects
2. **Docs** (9 chapters) — principles, architecture, agents, handoff schema, rules/skills, invocation modes, folder structure, common pitfalls (now 16), runbook
3. **Prompts** (3) — inventory, bootstrap (now with mandatory pre-flight safety checks), refinement
4. **Templates** (9) — drop-in agent files, schema, INDEX, spoonfeeder, archive README, rule, skill

Revision 4 changes: the BOOTSTRAP-PROMPT now includes mandatory pre-flight safety checks (snapshot, naming-collision detection, applies_to glob conflict detection, drift detection, decision gate) so the framework safely coexists with existing Claude Code configuration. New Pitfall 16 documents the bootstrap-on-existing-config scenario. Step 9 (CLAUDE.md merge) now requires a 3-pane diff before writing.

The most actionable single chapter is **Chapter 9 — Runbook**.

README

Claude Orchestration Framework

> **Purpose.** A reusable multi-agent orchestration setup for Claude Code that prevents cascading hallucinations, enforces evidence-based handoffs between agents, and makes Claude usable on production codebases by teams. Tech-stack agnostic — drops into any project (web, mobile, backend, ML, infra) in 2-4 hours.

Who this is for

- Engineering teams using Claude Code on real production codebases
- Projects with multiple roles (admin / customer / partner / etc.), multiple data systems, or compliance/security/payments concerns
- Anyone who has experienced Claude flooding context, drifting on rules, or silently propagating hallucinations across hops

If your project is a solo prototype or under ~5,000 lines, you probably don't need this yet — default Claude Code is enough. Adopt this when complexity passes the threshold.

The 30-second pitch

Default Claude Code is excellent for small tasks. For complex projects, you need:

- **One orchestrator that delegates to specialists** (instead of one agent doing everything and flooding context)
- **Bidirectional structured handoffs with evidence-bound claims** (instead of free-form prompts that propagate hallucinations)
- **Per-agent runtime constraints** — REVIEW-ONLY agents physically can't write; tool allowlists scope MCP access; `maxTurns` caps bound execution
- **A `failure_condition` field** that forces specialists to STOP if the orchestrator's premise turns out wrong (instead of pushing through on a false hypothesis)
- **Three-tier docs** — orientation maps for Claude, canonical refs for humans, frozen archive for history

This framework gives you all of that without runtime hooks or external dependencies — just Claude Code's built-in subagent features (`tools`, `maxTurns`, `Agent(...)`, `mcpServers`) plus documentation conventions.

What's in the box

```
claude-orchestration-framework/
├── README.md                          ← this file
├── Claude-Orchestration-Framework.pdf ← consolidated 57-page printable
├── LICENSE
│
├── docs/                              ← the framework explained (9 chapters)
│   ├── 01-PRINCIPLES.md               ← seven core principles
│   ├── 02-ARCHITECTURE.md             ← .claude/ + docs/ layout (with monorepo / mobile+web / multi-product variants)
│   ├── 03-AGENTS-GUIDE.md             ← how to design orchestrator + specialists (per-stack tables)
│   ├── 04-HANDOFF-SCHEMA.md           ← bidirectional schema + worked examples (web / mobile / REVIEW-ONLY)
│   ├── 05-RULES-AND-SKILLS.md         ← path-globbed rules + repeatable workflows
│   ├── 06-INVOCATION-MODES.md         ← claude vs --agent vs specialist mode
│   ├── 07-FOLDER-STRUCTURE.md         ← three-tier doc organization
│   ├── 08-COMMON-PITFALLS.md          ← 15 hard-won lessons
│   └── 09-RUNBOOK.md                  ← step-by-step bootstrap (~2-4 hours)
│
├── prompts/                           ← ready-to-paste prompts for Claude Code
│   ├── INVENTORY-PROMPT.md            ← scan + propose specialists (run first)
│   ├── BOOTSTRAP-PROMPT.md            ← generate all framework files (run second)
│   └── REFINEMENT-PROMPT.md           ← post-bootstrap hardening (run after 2-3 real tasks)
│
└── templates/                         ← drop-in templates with placeholders
    ├── orchestrator-agent.md.template
    ├── specialist-agent.md.template
    ├── review-only-agent.md.template
    ├── HANDOFF_SCHEMA.md.template
    ├── INDEX.md.template
    ├── SPOONFEEDER.md.template
    ├── rule.md.template
    ├── skill.md.template
    └── archive-README.md.template
```

Quick start — pick your scenario

Scenario A — Brand new project (greenfield)

```
# 1. Install Claude Code (if not already)
#    https://docs.claude.com/en/docs/claude-code

# 2. Open Claude Code in your new project root
cd ~/path/to/new-project
claude

# 3. Paste the BOOTSTRAP prompt directly (no inventory needed for greenfield)
#    Open prompts/BOOTSTRAP-PROMPT.md, copy contents, paste into Claude Code,
#    replace <framework path> with the absolute path to this repo on your machine
```

For greenfield, you can skip the inventory step and tell Claude what specialists you want directly:

```
Set up the Claude Orchestration Framework for a new <Next.js + Postgres + Stripe>
project named "<your-project-name>".

Specialists I want:
  - <project-slug>-orchestrator
  - frontend-ui
  - backend-api
  - database
  - payments
  - auth-security
  - qa-functional
  - release-devops
  - security-privacy (REVIEW-ONLY)

Use templates from <absolute path to this repo>/templates/.
Show me each generated file before saving.
```

Estimated time: **45 min - 1 hour**.

Scenario B — Existing project (brownfield)

⚠ **If your project ALREADY has Claude Code configuration** (e.g. you ran /init, or your team has been adding to CLAUDE.md for months), the bootstrap will detect this and ask before overwriting. The framework includes mandatory pre-flight safety checks:

- **Pre-flight 1** — auto-snapshot existing config to .claude-pre-bootstrap-backup/
- **Pre-flight 2** — naming-collision check (per file — STOP for explicit user decision)
- **Pre-flight 3** — applies_to: glob conflict check
- **Pre-flight 4** — drift detection on existing CLAUDE.md
- **Pre-flight 5** — existing agent style detection
- **Decision gate** — STOP if any pre-flight raised a <NEEDS USER CONFIRMATION> flag

You can also do an extra manual snapshot before starting:

```
mkdir -p .claude-pre-bootstrap-backup
[ -f CLAUDE.md ] && cp CLAUDE.md .claude-pre-bootstrap-backup/
[ -d .claude ] && cp -r .claude .claude-pre-bootstrap-backup/
```

The framework's BOOTSTRAP-PROMPT will repeat the snapshot inside the prompt — this is just belt-and-suspenders.

```
# 1. Install Claude Code (if not already)

# 2. Open Claude Code in the existing project's root
cd ~/path/to/existing-project

# 3. Make sure git is clean — bootstrap creates new files; you want a clean baseline
git status
git checkout -b setup/claude-orchestration

# 4. Run the inventory pass (read-only — proposes what to build)
claude
# Paste: prompts/INVENTORY-PROMPT.md
# Replace <framework path> with the absolute path to this repo on your machine
# INVENTORY scans for existing CLAUDE.md, .claude/agents/, .claude/rules/, .claude/skills/

# 5. Review/adjust Claude's proposed inventory + answer Open Questions

# 6. Run the bootstrap pass (creates all framework files)
# Paste: prompts/BOOTSTRAP-PROMPT.md (in the same Claude Code session)
# BOOTSTRAP runs pre-flight safety checks BEFORE creating any files
# If existing config is detected, you'll be asked to confirm per-file decisions

# 7. Verify (in terminal)
claude agents                # should list all your project specialists
[ -f .claude-pre-bootstrap-backup/CLAUDE.md ] && diff .claude-pre-bootstrap-backup/CLAUDE.mc
npm run build                # or whatever your build is — should still pass

# 8. Run a real task via the orchestrator
claude --agent <project-slug>-orchestrator
# Give it a real bug or small feature; verify the handoff schema works end-to-end

# 9. Commit and PR (note: .claude-pre-bootstrap-backup/ is gitignored)
git add .claude/ docs/ai-context/ docs/_archive/ CLAUDE.md .gitignore
git commit -m "chore: bootstrap Claude Code orchestration framework"
git push -u origin setup/claude-orchestration
gh pr create --base <your-base-branch> --title "Bootstrap Claude orchestration framework"
```

Estimated time: **2-4 hours** (split across phases — see docs/09-RUNBOOK.md).

Scenario C — Just want to read the framework

Read the **PDF** (Claude-Orchestration-Framework.pdf) — 57 pages, fully self-contained. Or browse the markdown files in docs/ for clickable cross-links.

The most actionable single chapter is **docs/09-RUNBOOK.md**.

Prerequisites

- **Claude Code** installed and authenticated. Verify with claude --version (need 2.x or later). Install guide.
- **Git** installed (any recent version).
- **Optional but recommended:** GitHub CLI (gh) for PR creation.

No other dependencies. The framework is markdown + Claude Code's built-in subagent features.

What this framework does NOT include

- **Runtime hooks.** Documentation enforcement covers ~95% of value at 5% of complexity. Hooks are added later (per docs/08-COMMON-PITFALLS.md recommendations) IF documentation enforcement demonstrably fails.
- **Code generation.** This is purely a documentation + agent-config framework. Your application code stays untouched.
- **Vendor lock-in.** No external services, no SaaS, no API keys. Just markdown.
- **Project-specific specialists.** The templates have placeholders. You customize per project.

Customizing the framework for your project

Three things change per project:

1. **Project name + slug** (used to name the orchestrator: <slug>-orchestrator)
2. **Specialist list** (5-10 specialists matching your project's domain boundaries)
3. **Path globs in rules** (matched to your actual code paths)

Everything else (the handoff schema, the universal evidence rule, the failure_condition pattern, the three-tier docs structure, the invocation modes) stays the same across projects.

See **docs/03-AGENTS-GUIDE.md** for how to pick your specialist list and **docs/05-RULES-AND-SKILLS.md** for how to write rules.

Maintenance

After bootstrapping, the framework needs minimal upkeep:

- **Per task:** rules accumulate naturally as production teaches you new gotchas (add 1-2 per month is normal)
- **Per quarter:** run prompts/REFINEMENT-PROMPT.md to audit specialist scope drift, archive stale docs, and decide if hooks/memory should be added
- **Per major refactor:** the context-librarian specialist (if you spawned one) handles docs reorganization

Where this came from

Battle-tested on a production K-12 codebase with: - 11 specialist agents (frontend, backend, real-time video, payments, security, legal-compliance, QA, DevOps, AI/ML pipeline, product-flow, docs) - 7 path-globbed rule files - 6 repeatable workflows - Three-tier doc organization (~21 orientation maps + 17 canonical refs + frozen archive) - Bidirectional handoff schema enforced via specialist refusal contracts

The cascading-hallucination defense (failure_condition) and the universal evidence rule are the framework's two biggest novel contributions beyond stock Claude Code.

Frequently asked

Q: Is this Anthropic-official? No. This is a community framework built on top of Claude Code's documented subagent features. No special access required.

Q: Will it work with the Claude API directly (not Claude Code)? The orchestrator-worker pattern and handoff schema concepts translate, but the runtime enforcement (tools, maxTurns, Agent(...) allowlists) is Claude Code specific. For pure API usage, you'd need to implement those with hooks or middleware.

Q: Does it work with Cursor / GitHub Copilot / other AI assistants? The documentation patterns are portable, but tools allowlists and subagent isolation aren't. You'd lose most of the runtime enforcement layer.

Q: Can I use this on a monorepo? Yes. See docs/02-ARCHITECTURE.md § "Variations" for monorepo, multi-product, and mobile+web split patterns.

Q: I'm worried about leaking sensitive code. The framework doesn't change Claude Code's data handling. Use Claude Code's permissions.deny rules for sensitive paths. See docs/09-RUNBOOK.md § "What if I'm worried about leaking sensitive code".

Q: Can I share this with another engineer who isn't on my team? This repo is private. Add them as a collaborator on GitHub, or zip the folder and email it. The framework itself is free to use — see LICENSE.

License

MIT (see LICENSE). Use freely. Adapt freely. Attribution appreciated but not required.

Contributing back

If you discover a missing pitfall, a better template pattern, or a useful new prompt, edit the markdown files and commit. The framework improves as it sees more projects.

If you've used it on a project and want to share what worked / didn't, open an issue on this repo with notes — useful for evolving the templates.

ewpage

01 — Principles

The seven principles this framework rests on. Every other doc, prompt, and template implements these.

1. Orchestrator-worker pattern, documented not

coded

One coordinator agent reads tasks, classifies them, picks the right specialists, and aggregates findings. Specialists do focused work in isolated context windows.

The orchestration is **prescriptive documentation**, not runtime code. The "orchestrator" is a markdown file with instructions; the harness (Claude Code) provides isolation, tool restrictions, and turn caps. There is no orchestration service, no message queue, no central process.

This matters because: - It works with any LLM that supports subagents - It's fully transparent — you can read every rule - It's reversible — delete .claude/ and you're back to default - Hard runtime enforcement (hooks, settings.deny) can be added later as a separate hardening pass

2. Specialists run in isolated context

Each specialist agent runs in its own context window. It does not see the orchestrator's full conversation. It receives a focused prompt (the handoff) and returns a focused result (the return).

This is the **single most important defense against context flooding**. A specialist investigating a database bug doesn't need to know about your unrelated UI redesign in another open thread. Isolation makes each specialist's reasoning sharp and their output auditable.

3. Universal evidence rule — "no evidence, no claim"

If a claim cannot be tied to a file:line, table:column, rule, doc anchor, test, or command output, it must be marked as an assumption (confidence: low outbound, or moved to unverified_claims inbound) and cannot be passed downstream as fact.

This rule appears verbatim in: - The handoff schema (both directions) - The orchestrator's "outbound discipline" - Every specialist's "incoming handoff validation"

It is the **core defense against cascading hallucinations** across hops. An orchestrator that hallucinates "the bug is in foo.ts:42" must cite that path; the specialist re-verifies before editing; if wrong, the specialist returns status: claim_rejected with the evidence of what it actually found.

4. Failure condition — articulate what would prove you wrong

Every outbound handoff includes a failure_condition field: one sentence stating what observable evidence would prove the orchestrator's hypothesis or delegation premise wrong. If the specialist observes that condition, it stops, returns claim_rejected, and includes the triggering evidence.

This is the **inverse of verify_before_acting.** Where verify_before_acting says "check these before acting," failure_condition says "if you observe this, STOP." Together they bracket the work in falsifiable claims.

Forcing the orchestrator to articulate failure_condition is itself a thinking discipline — if you can't name what would falsify your hypothesis, you don't have a hypothesis, you have a guess.

5. Tools and turns are runtime-enforced; everything else is discipline

The Claude Code harness enforces: - tools: allowlists (specialist physically cannot use tools not listed) - disallowedTools: denylists (subtract from inheritance) - maxTurns: (hard cap on agent execution) - Agent(name1, name2, ...) allowlists (orchestrator can only spawn allowlisted specialists) - mcpServers: per-agent MCP server scoping - Subagent context isolation

Everything else (handoff schema fields, refusal behavior, evidence rule, hop limits, "rules read before edit") is **documentation discipline** — relies on agents following their instructions. This is acceptable because: - Subagents are isolated, so a misbehaving agent's damage is bounded - The orchestrator catches schema violations on receipt of a return - A poorly-documented expectation can be tightened into a hook later if needed

Don't conflate the two layers. When you say "the orchestrator only delegates to project specialists," that's enforced at runtime via Agent(name1, name2, ...). When you say "specialists refuse vague delegations," that's documented in their instructions

and only as enforced as the agent's compliance.

6. Three-tier documentation

Project documentation should split into three clearly-labeled tiers:

Tier	Location	Purpose	Read by
Orientation maps	docs/ai-context/<topic>.md	50-150 lines per area; gotchas; cross-links to canonical	Agents (per task routing)
Canonical references	docs/<UPPERCASE>.md	Architecture, API, schema, business — full detail	Humans + agents (when deeper)
Frozen archive	docs/_archive/<date>/	Sprint reports, post-mortems, migration logs, snapshots	Audits only — never linked from active docs

This separation makes drift impossible. Active docs live in the first two tiers. The archive is append-only and never referenced by agents/rules/active docs. When something is no longer authoritative, it moves to archive — it doesn't sit in docs/ confusingly mixed with truth.

7. Default Claude Code stays default

Do not make the orchestrator the project default. Setting agent: "<orchestrator-name>" in .claude/settings.json would force every session into orchestrator mode, which means: - No Edit/Write tools (every code change requires Agent(specialist) delegation — heavy friction for typos) - A maxTurns cap restrictive for interactive iterative work - Default Claude Code system prompt replaced entirely by the orchestrator's body prompt

The orchestrator is for **medium/complex tasks** — explicitly invoked via claude --agent <orchestrator-name>. Plain claude stays available for everyday work. Specialists can also be invoked directly via claude --agent <specialist-name> for narrow domain work.

Three invocation modes coexist by design. See 06-INVOCATION-MODES.md.

What these principles get you

A team can run dozens of parallel sessions on the same project without context collision. Specialists never inherit baggage from unrelated work. Orchestrator decisions are auditable (every claim cites evidence). Hallucinations don't compound across hops. New engineers see a clean folder layout that tells them where to look.

The cost is upfront discipline: you spend a day writing the agent contracts, rules, and orientation maps. The payback is permanent — every future task benefits.

ewpage

02 — Architecture

The physical layout of files in a project that uses this framework.

The full picture

```
your-project/
│
├── CLAUDE.md                          ← project router (auto-loaded by Claude Code)
│
├── .claude/
│   ├── agents/                        ← orchestrator + N specialists
│   │   ├── <project>-orchestrator.md
│   │   ├── <specialist-1>.md
│   │   ├── <specialist-2>.md
│   │   └── ...
│   ├── rules/                         ← path-globbed invariants
│   │   ├── <domain-1>.md
│   │   ├── <domain-2>.md
│   │   └── ...
│   ├── skills/                        ← repeatable workflows
│   │   ├── <workflow-1>/SKILL.md
│   │   └── ...
│   ├── settings.json                  ← project-wide Claude Code settings
│   └── settings.local.json            ← personal permissions allowlist (gitignored if
sensitive)
│
├── docs/
│   ├── ai-context/                    ← orientation maps (read by agents)
│   │   ├── INDEX.md                   ← task → docs router
│   │   ├── HANDOFF_SCHEMA.md          ← bidirectional handoff schema
│   │   ├── ORCHESTRATION_SPOONFEEDER.md  ← human usage guide
│   │   └── <area>-experience.md       ← per-domain orientation, 50-150 lines each
│   ├── ARCHITECTURE.md                ← canonical refs (full detail)
│   ├── API.md
│   ├── <YOUR-OTHER-CANONICAL-DOCS>.md
│   └── _archive/                      ← frozen historical
│       ├── README.md                  ← archive convention
│       └── <YYYY-MM>/                 ← dated reports, post-mortems
│
└── <your application code>/
```

What lives where, and why

CLAUDE.md (root)

The router. Auto-loaded into every Claude Code session in this project. Should be **under 200 lines**. Contains: - Identity (one sentence about the project) - Golden rules (security, deployment, etc.) - Mandatory workflow (numbered steps for every task) - Routing tables (task → doc, task → agent) - Cross-links to the spoonfeeder, INDEX, and canonical docs

Does NOT contain: - Long technical detail (lives in canonical docs) - Per-domain gotchas (lives in .claude/rules/*.md) - Sprint history or audit content (lives in docs/_archive/)

.claude/agents/

One markdown file per agent. Frontmatter declares name, description, tools, maxTurns, optionally model/memory/etc. Body declares the agent's contract: when to use, when not, required reading, I CAN, I CANNOT, definition of done, plus the handoff validation + return schema sections.

Always includes exactly one orchestrator. Specialists are added based on the project's natural domain boundaries (typically 5–12 specialists for a medium-complexity codebase).

See 03-AGENTS-GUIDE.md for how to design agents.

.claude/rules/

Path-globbed invariants. Each rule file's frontmatter has applies_to: listing path globs. When an agent is about to edit a file matching one of those globs, it must read the rule first. The rule contains "hard rules," "what not to do," and cross-links.

Example: .claude/rules/database.md might apply to src/db/**/*.ts and contain rules like "never run raw SQL in route handlers; use the repository layer."

Rules carry the **hard-won gotchas** of your codebase — things that broke production once and must not break again. They are short, scoped, and grow over time.

.claude/skills/

Each skill is a directory with a SKILL.md file inside. The skill is a **repeatable workflow** — a template for tasks that recur. Examples: - investigate-bug/SKILL.md — bug investigation steps - build-feature/SKILL.md — feature build steps - qa-flow/SKILL.md — pre-release QA pass - compliance-review/SKILL.md — review for regulatory risks

Skills are invoked by name when the orchestrator (or a human) decides this task fits a known pattern.

docs/ai-context/

Orientation maps. Each file is 50-150 lines and covers one area (e.g. auth-and-sessions.md, payments.md, realtime-sync.md). Format: - Top: 2-sentence orientation - Middle: key file paths + table list + the 3-5 gotchas worth knowing - Bottom: cross-links to canonical refs

These are what agents read when deciding "what do I need to know about before touching code in it." Cheaper than reading 1000-line canonical docs.

Special files in docs/ai-context/: - INDEX.md — the master router (task → docs + rules + agent map) - HANDOFF_SCHEMA.md — the bidirectional handoff schema (this framework's central artifact) - ORCHESTRATION_SPOONFEEDER.md — the human-facing usage guide - MIGRATION_LEDGER.md (optional) — tracks doc/structure migration progress - LEGACY_BACKUP.md (optional) — frozen pre-refactor snapshot

docs/<UPPERCASE>.md

Canonical references. Full detail. Examples: - ARCHITECTURE.md — system architecture - API.md — endpoint reference - TECH_STACK.md — what's installed and why - BUSINESS_DOCUMENT.md — non-technical product/business notes - CHANGELOG.md — release log

These don't change file layout per-task; they're the encyclopedic source of truth.

docs/_archive/

Frozen historical material. Sprint reports, post-mortems, dated audits, migration logs. Two rules: 1. Active docs (CLAUDE.md, docs/ai-context/*, agents, rules) **never link into _archive/**. 2. The archive is append-only — date subdirs, never reorganize within.

The docs/_archive/README.md documents these conventions for future contributors.

Why this layout works

Predictability for Claude

Every agent knows where to find: - Its own contract: .claude/agents/<self>.md - The handoff schema: docs/ai-context/HANDOFF_SCHEMA.md - Routing: docs/ai-context/INDEX.md - Rules for paths it might edit: .claude/rules/*.md matched via applies_to - Canonical refs: docs/<UPPERCASE>.md

There is no ambiguity. New tasks follow the same path every time.

Predictability for humans

A new engineer joining the project sees: - CLAUDE.md at root — tells them how Claude Code is set up - docs/ — where documentation lives - .claude/ — Claude-specific config - Application code in its standard tech-stack location

They can ignore .claude/ if they don't use Claude Code. The framework adds value without imposing tax on humans who prefer their own tooling.

Minimal coupling to tech stack

Nothing in .claude/ or docs/ai-context/ cares about your framework, language, or deploy target. The agents reference paths in your actual codebase, but the agent FILES themselves are pure markdown. You can reorganize your codebase entirely and only need to update the rule globs and the orientation maps' cross-links.

What does NOT belong in this framework

- **Application code.** Stays where your tech stack puts it.
- **Build config.** Stays at root (your package.json, pyproject.toml, Cargo.toml, go.mod, etc.).
- **CI workflows.** Stays in .github/, .gitlab-ci.yml, etc.
- **Test files.** Stays in your tests directory.
- **Generated artifacts.** Gitignored.

The framework is **purely additive** to whatever else lives in your repo.

Variations

Monorepos

For monorepos, you can either: - Have one .claude/ at the root that knows about all packages, or - Have one .claude/ per package + a root .claude/ that orchestrates across packages

The first is simpler. The second is more isolated. Pick based on whether tasks routinely cross package boundaries.

Mobile + web split

If your project has separate mobile and web codebases in one repo, your orchestrator typically delegates to a mobile-ui specialist OR a web-ui specialist depending on the task. They can share a backend-api specialist for the API-side work.

Multi-tenancy / multi-product

If you serve multiple products from one codebase, consider per-product orientation maps and per-product rules. The orchestrator can be one or per-product depending on whether tasks routinely cross products.

Heavy infra / DevOps focus

Add specialists for infrastructure, observability, release-automation. Their rule files cover Terraform/CDK/Kubernetes/Helm conventions.

ML / data heavy

Add specialists for data-pipeline, model-training, inference-serving. Their orientation maps cover dataset locations, experiment tracking, model versioning.

The architecture flexes; the principles don't.

ewpage

03 — Agents Guide

How to design the orchestrator and specialists for your project.

The orchestrator

Every project has exactly **one** orchestrator. Naming convention: <project-name>-orchestrator (e.g. acme-orchestrator, glowgrades-orchestrator).

Responsibilities

- Read incoming task. Restate it.
- Identify affected roles / domains.
- Read minimum docs (INDEX.md → 1-3 orientation maps).
- Pick the right specialists (typically 1-3 per task).
- Issue structured handoffs with claims + evidence + failure_condition.
- Receive returns. Verify rejected_claims. Re-verify unverified_claims before propagating.
- Aggregate findings into a final summary with the project's "Definition of Done."

Constraints

- **No Edit/Write tools.** The orchestrator coordinates; specialists implement.
- **Restricted Agent allowlist.** Only project specialists, not built-ins or plugins.
- **Modest maxTurns.** 20-30 turns is typical (read INDEX → delegate 1-3 times → aggregate).

Frontmatter template

```
---
name: <project>-orchestrator
description: Main task dispatcher for <Project>. Use for medium/complex tasks. Picks minimum dc
tools: Read, Grep, Glob, Bash, WebFetch, WebSearch, TodoWrite, Agent(<specialist1>, <specialis
maxTurns: 25
---
```

Specialists

A specialist owns one domain. Specialists are domain-shaped, not technology-shaped. Bad: react-specialist, typescript-specialist. Good: frontend-ui, payments, realtime-

sync, compliance.

How many do you need?

Project size	Specialist count
Solo project / prototype	0-2 (just use plain claude)
Small team / single product	4-6
Medium codebase / multiple roles	6-10
Large codebase / multi-product	10-15

More than 15 is a smell — you're probably making them too narrow.

Common specialists by project type

Web app (any stack)

Specialist	Domain
frontend-ui	UI components, routing, state management, client-side interactions
backend-api	API routes, server logic, request handling
database	Schema, migrations, query layer, indexes
auth-security	Authentication, authorization, sessions, secrets
qa-functional	Browser/E2E testing, regression checks
release-devops	Build, deploy, env config, observability

Mobile app

Specialist	Domain
mobile-ui	Screens, navigation, gestures, platform-specific UX
mobile-state	State management, persistence, offline sync
mobile-native	Platform bridges (iOS/Android native code or Flutter plugins)
backend-api	Server-side endpoints the app calls
qa-mobile	Device testing, simulator/emulator flows

Backend / API service

Specialist	Domain
api-routes	HTTP/gRPC route handlers
domain-logic	Core business logic
data-layer	DB access, repositories, ORM patterns
integrations	Third-party API clients
observability	Logging, metrics, tracing
release-devops	Deploy, env, secrets

ML / data project

Specialist	Domain
data-pipeline	ETL, ingestion, transformation
model-training	Training scripts, hyperparam tuning, experiment tracking
inference-serving	Model serving, latency, batching
data-quality	Validation, anomaly detection, monitoring

Cross-cutting (add to most projects)

Specialist	Domain
product-flow	Acceptance criteria, cross-role behavior, requirement clarification (LIGHT EDITS ONLY)
qa-functional	Tests across the system
security-privacy	Auth/RLS/sensitive-data review (REVIEW-ONLY)
legal-compliance	Regulatory review for relevant laws (REVIEW-ONLY)
release-devops	Build/deploy/cron/env
context-librarian	Maintains .claude/, docs/ai-context/, archives drift
docs-writer	Canonical doc updates

REVIEW-ONLY specialists

Some specialists should explicitly **not** edit code. Tag them in the description, give them no Edit/Write tools, and document the constraint in their I CANNOT section.

Examples: - legal-compliance — flags risks; never claims final legal advice - security-privacy — identifies vulnerabilities; doesn't ship fixes - architecture-review — reviews proposed designs; doesn't implement

Frontmatter for REVIEW-ONLY:

```
---
name: legal-compliance
description: REVIEW-ONLY. Flags <list of regulations> risks. Produces attorney-checklist content.
tools: Read, Grep, Glob, WebFetch, WebSearch, TodoWrite
maxTurns: 12
---
```

Notably absent: Edit, Write, Bash. The harness physically prevents this agent from modifying files.

⚠ **Do NOT enable memory: on REVIEW-ONLY agents.** Per Claude Code docs, enabling memory: automatically grants Read/Write/Edit so the agent can manage its memory files — silently bypassing your REVIEW-ONLY contract.

Implementation specialists

Get full Edit/Write/Bash access plus relevant MCP tools.

Frontmatter for an implementation specialist:

```
---
name: backend-api
description: Builds and fixes API routes, server logic, request handling. Coordinates with database
tools: Read, Edit, Write, Grep, Glob, Bash, TodoWrite
maxTurns: 20
---
```

For specialists that need MCP server access (e.g. browser testing), use the documented wildcard:

```
tools: Read, Edit, Write, Grep, Glob, Bash, TodoWrite, mcp__chrome-devtools__*, mcp__playwrig
```

The wildcard is supported and covers all current + future tools from that MCP server.

Agent body structure (every agent)

Every agent file body should include these sections in order:

```
# <Agent Display Name>

<2-sentence description of the agent's domain.>

## When to use

- Bullet list of task types this agent owns

## When NOT to use

- Bullet list of task types that belong to other agents

## Required reading

1. `docs/ai-context/INDEX.md` — task → docs map
2. <area-specific orientation map>
3. **`.claude/rules/<your-rule>.md`** — always (when applicable)
4. <other rules / canonical refs>

## Incoming handoff validation

<paste the standard incoming handoff validation block — see template>

## Return schema (required)

<paste the standard return schema block — see template>

## I CAN

- Bullet list of what this agent is allowed to do

## I CANNOT

- Bullet list of what this agent must refuse
- Always include: "Push to main" (or your equivalent protected branch)

## Definition of Done

1. Files changed (paths)
2. Rule files read before editing
3. Tests run
4. Risks remaining
5. (any agent-specific Definition of Done items)

## Cross-links

- `<related rule>`
- `<related orientation map>`
- `<canonical doc>`
```

The “Incoming handoff validation” and “Return schema” sections are **identical across all specialists** — they enforce the universal handoff contract. See templates/specialist-agent.md.template.

Naming conventions

- Lowercase with hyphens: backend-api, frontend-ui, legal-compliance
- Filename matches name: backend-api → backend-api.md
- Be specific but not over-narrow: payments good, stripe-webhook-handler too narrow
- Cross-cutting concerns get their own agent: qa-functional, security-privacy, not folded into another specialist

When to add a new specialist vs. extend an existing one

Add a new specialist when: - The domain has its own gotchas (would warrant its own rule file) - Tasks in that domain frequently arrive - The existing specialists’ “I CANNOT” sections would all reject this work

Extend an existing specialist when: - The work overlaps with an existing specialist’s domain - The new domain is small (fits in 1-2 paragraphs of the existing agent’s body) - You haven’t seen 3+ tasks in this domain yet

When in doubt: don’t add. It’s easier to split a specialist later than to delete one that accumulated cruft.

Anti-patterns

Technology-named specialists. typescript-specialist doesn’t have a clear domain — what does it own that frontend-ui and backend-api don’t already cover?

Layer-named specialists. database-specialist is fine if “database” means a

domain (schema, migrations, repository layer); not fine if it means “anyone touching SQL anywhere should consult me.”

Specialists with overlapping `tools` and overlapping descriptions. Confuses the orchestrator; it won’t know which to pick.

Specialists with no `tools:` field. They inherit everything, defeating defense-in-depth. Always declare `tools` explicitly.

Specialists that delegate to other specialists. Per Claude Code semantics, subagents cannot spawn subagents. If a specialist needs another specialist’s input, it returns to the orchestrator with `recommended_next_agent` and lets the orchestrator chain the work.

Specialists whose “I CAN” includes “approve production deploys.” That’s the project owner’s call, not an agent’s.

ewpage

04 — Handoff Schema

The bidirectional schema for orchestrator ↔ specialist communication. This is the framework’s central artifact.

Why a schema

Without structure, handoffs are free-form prompts. Specialists guess what’s expected. Orchestrators don’t commit to specific claims. Cascading hallucinations compound across hops because nothing forces evidence-to-claim binding at the boundary.

The schema closes both gaps with two YAML blocks (one per direction) and one universal evidence rule.

Universal evidence rule

> **No evidence, no claim.** If a claim cannot be tied to a file, line, table, column, rule, doc, test, or command output, it must be marked as an assumption (`confidence: low` outbound, or moved to `unverified_claims` inbound) and cannot be passed downstream as fact.

This rule applies to both directions of every handoff. It is repeated verbatim in the orchestrator’s outbound-discipline section and in every specialist’s incoming-handoff-validation section.

Outbound: orchestrator → specialist

Every Agent-tool delegation from the orchestrator MUST include this YAML block at the top of the prompt.

```yaml
handoff:
  schema_version: 1
  handoff_id: <short unique id, e.g. h-2026-05-02-a3f>
  from_agent: <orchestrator-name>
  to_agent: <specialist-name>
  goal: <one sentence — what done looks like>
  task_class: <bug | feature | review | qa | refactor | audit>
  affected_roles: [<role1>, <role2>, ...]
  claims:
    - claim: <factual statement the orchestrator commits to>
      evidence: <path/to/file.ts:42 OR rule:.claude/rules/foo.md OR doc:docs/ARCHITECTURE.md#
      confidence: <high | medium | low>
  verify_before_acting:
    - <claim or assumption the specialist MUST re-verify against source before editing>
  failure_condition: <one sentence — observable evidence that proves the orchestrator's hypothes
  in_scope:
    - <path or component the specialist may edit>
  out_of_scope:
    - <path or component the specialist must NOT touch — even if "while we're in there">
  rules_to_read:
    - <.claude/rules/*.md path the specialist must read first>
  expected_artifacts:
    - <file changed, test added, summary section, etc.>
  hop_context:
    parent_task: <one line — why the orchestrator is delegating>
    previous_specialist: <agent name or "none">
    previous_findings_summary: <one line — what came back, or "n/a">
```

Field requirements (outbound)

Field	Required	Notes
schema_version	yes	Currently 1. Bump on breaking changes.
handoff_id	yes	Short unique id. Specialist echoes it on return so they pair.
from_agent	yes	Always the orchestrator's name.
to_agent	yes	The specialist's name field.
goal	yes	One sentence with measurable outcome. "Fix" or "improve" without an outcome is too vague.
task_class	yes	One of the six values.
affected_roles	yes	Non-empty list.
claims	yes (may be empty list [])	Every entry MUST have evidence. Use confidence: low for assumptions.
verify_before_acting	yes if task touches code	Empty allowed only for pure-research handoffs.
failure_condition	yes	One sentence. The inverse of verify_before_acting. Use failure_condition: open-ended exploration; no specific premise to invalidate only when no premise truly exists.
in_scope	yes	Non-empty list.
out_of_scope	yes	Empty list [] is valid. Missing key is not.
rules_to_read	yes	Use ["specialist will discover from path globs"] if you don't know which rules apply.
expected_artifacts	yes	Drives the specialist's files_changed / tests_run later.
hop_context	yes	All three sub-fields required. Use none / n/a for first hop.

Inbound: specialist → orchestrator (return schema)

Every specialist response MUST end with this YAML block.

```yaml
return:
  schema_version: 1
  handoff_id: <copy from inbound — pairs the response>
  from_agent: <specialist-name>
  to_agent: <orchestrator-name>
  status: <completed | blocked | needs_clarification | claim_rejected>
  verified_claims:
    - claim: <orchestrator claim that was confirmed against source>
      evidence: <path:line or command output the specialist saw>
  rejected_claims:
    - claim: <orchestrator claim that turned out to be wrong>
      evidence: <what the specialist actually found>
  unverified_claims:
    - claim: <orchestrator claim that was not checked>
      reason: <why — out of scope, infra not available, etc.>
  evidence_used:
    - <path:line, rule, doc anchor, or command output>
  files_changed:
    - <path:line range and one-line summary>
  tests_run:
    - <command + result>
  risks:
    - <risk + mitigation or "none">
  recommended_next_agent:
    agent: <specialist-name or "none">
    why: <one line>
  do_not_pass_downstream_without_verification:
    - <claim or finding the orchestrator must NOT propagate as fact>
```

Field requirements (inbound)

Field	Required	Notes
schema_version	yes	Must match outbound.
handoff_id	yes	Must echo inbound id verbatim.
from_agent / to_agent	yes	Self-explanatory.
status	yes	`completed` only if all `verify_before_acting` items checked. Use `claim_rejected` when an orchestrator claim was wrong.
verified_claims	yes (may be [])	Empty list is valid. Missing key is not.
rejected_claims	yes (may be [])	Empty list is valid. Non-empty implies `status: claim_rejected` or `blocked`.
unverified_claims	yes (may be [])	Anything you accepted without re-checking goes here.
evidence_used	yes	Real paths/commands. “I read the codebase” is not evidence.
files_changed	yes (may be [])	Empty list valid for review/audit handoffs.
tests_run	yes (may be [])	Empty list valid for pure-doc changes.
risks	yes	Use `["none"]` if there are genuinely none.
recommended_next_agent	yes	`agent: "none"` if work is complete.
do_not_pass_downstream_without_verification	yes (may be [])	Cascading-hallucination defense. Be liberal.

Refusal

If the inbound handoff fails the field requirements, the specialist responds with ONLY this block, does no other work, and waits for re-issue:

```
return:
  schema_version: 1
  handoff_id: <copy from inbound, or "missing" if absent>
  from_agent: <specialist-name>
  to_agent: <orchestrator-name>
  status: needs_clarification
  reason: handoff_schema_invalid
  missing_or_invalid_fields:
    - field: <field name>
      problem: <what's wrong — missing, vague, no evidence, etc.>
  required_action:
    - <what the orchestrator must add or fix before re-issuing>
```

Failure-condition observation rule

If at any point during the work the specialist observes the inbound `failure_condition`, it STOPS immediately and responds with the return schema using `status: claim_rejected` (or `needs_clarification` if ambiguous). It cites the triggering evidence in `evidence_used`. It does NOT push through a falsified premise — even if mid-edit, partial work has already been done, or the fix seems easy.

Worked examples (any tech stack)

Example 1 — Backend bug investigation

Outbound (orchestrator → backend-api):

```
handoff:
  schema_version: 1
  handoff_id: h-2026-05-02-bug-001
  from_agent: acme-orchestrator
  to_agent: backend-api
  goal: Find why /api/orders returns 500 for orders with > 100 line items.
  task_class: bug
  affected_roles: [customer]
  claims:
    - claim: The 500 originates in the order serialization step.
      evidence: src/api/orders/handler.ts:88
      confidence: high
    - claim: Order serialization runs synchronously and may exceed the function timeout.
      evidence: rule:.claude/rules/api.md
      confidence: medium
  verify_before_acting:
    - That src/api/orders/handler.ts:88 is still the serialization line.
    - That the timeout is actually exceeded (logs / metric, not just suspected).
  failure_condition: If logs show the 500 happens in the database query rather than serialization, S
  in_scope:
    - src/api/orders/handler.ts
    - src/api/orders/serializer.ts
  out_of_scope:
    - The order schema (database concern; separate specialist if migration needed).
  rules_to_read:
    - .claude/rules/api.md
    - .claude/rules/observability.md
  expected_artifacts:
    - Root cause statement (file:line)
    - Smallest fix or "no bug — load test issue" verdict
  hop_context:
    parent_task: Customer reports cart > 100 items always errors at checkout.
    previous_specialist: none
    previous_findings_summary: n/a
```

Return (backend-api → orchestrator):

```yaml
return:
  schema_version: 1
  handoff_id: h-2026-05-02-bug-001
  from_agent: backend-api
  to_agent: acme-orchestrator
  status: claim_rejected
  verified_claims:
    - claim: src/api/orders/handler.ts:88 is the serialization line.
      evidence: src/api/orders/handler.ts:88 (confirmed via grep + read)
  rejected_claims:
    - claim: The 500 originates in serialization.
      evidence: Stack trace from production logs shows the failure is in the database query at src/db
  unverified_claims: []
  evidence_used:
    - src/api/orders/handler.ts:80-95
    - src/db/repositories/orderRepo.ts:140-150
    - command:grep -rn "loadOrderLineItems" src/
    - production logs (sampled 3 failing requests)
  files_changed: []
  tests_run: []
  risks:
    - Real fix requires pagination of the query. Out of this handoff's scope.
  recommended_next_agent:
    agent: database
    why: Pagination of orderRepo.loadOrderLineItems is the correct fix.
  do_not_pass_downstream_without_verification:
    - That the timeout value is actually 30s (read from env or config; I inferred from observed behav
```

Example 2 — Mobile feature build

Outbound (orchestrator → mobile-ui):

```yaml
handoff:
  schema_version: 1
  handoff_id: h-2026-05-02-feat-002
  from_agent: acme-orchestrator
  to_agent: mobile-ui
  goal: Add a pull-to-refresh gesture on the OrderHistoryScreen.
  task_class: feature
  affected_roles: [customer]
  claims:
    - claim: OrderHistoryScreen lives at src/screens/OrderHistoryScreen.tsx.
      evidence: src/screens/OrderHistoryScreen.tsx
      confidence: high
    - claim: The app uses React Native's RefreshControl for pull-to-refresh elsewhere.
      evidence: rule:.claude/rules/mobile-ui.md
      confidence: high
  verify_before_acting:
    - That OrderHistoryScreen uses a ScrollView or FlatList that can host RefreshControl.
  failure_condition: If OrderHistoryScreen renders a non-scrollable view (e.g. a Map or a custom vi
  in_scope:
    - src/screens/OrderHistoryScreen.tsx
    - src/screens/__tests__/OrderHistoryScreen.test.tsx (add test)
  out_of_scope:
    - The order data fetching logic (already exists; just call refetch).
  rules_to_read:
    - .claude/rules/mobile-ui.md
  expected_artifacts:
    - Edit to OrderHistoryScreen.tsx adding RefreshControl
    - New test asserting onRefresh triggers refetch
  hop_context:
    parent_task: Product wants pull-to-refresh on order history per ACME-1234.
    previous_specialist: none
    previous_findings_summary: n/a
```

Example 3 — Compliance review (REVIEW-ONLY specialist)

Outbound (orchestrator → compliance):

```yaml
handoff:
  schema_version: 1
  handoff_id: h-2026-05-02-rev-003
  from_agent: acme-orchestrator
  to_agent: compliance
  goal: Review the new email opt-in flow for GDPR/CASL/CAN-SPAM compliance before launch.
  task_class: review
  affected_roles: [customer]
  claims:
    - claim: The opt-in flow records consent timestamp + IP + opt-in source.
      evidence: src/db/migrations/2026-05-01-add-consent-log.sql
      confidence: high
    - claim: Marketing emails currently send without an explicit unsubscribe link.
      evidence: src/email/templates/marketing.tsx:42
      confidence: medium
  verify_before_acting:
    - That the migration has been applied to staging.
    - That the consent log columns are actually written by the new opt-in handler.
  failure_condition: If the consent log table exists but the opt-in handler does NOT write to it, STOP
  in_scope:
    - REVIEW-ONLY. No code edits.
    - src/email/**
    - src/db/migrations/2026-05-01-*
  out_of_scope:
    - Any code changes (escalate to email-templates or backend-api specialists).
  rules_to_read:
    - .claude/rules/security-privacy.md
  expected_artifacts:
    - Severity-ranked findings (blocker / high / medium / low)
    - Attorney-checklist of questions for counsel
  hop_context:
    parent_task: Pre-launch gate for new email opt-in feature.
    previous_specialist: none
    previous_findings_summary: n/a
```

Versioning

- `schema_version: 1` is current.
- Additive changes (new optional fields) keep version 1.
- Breaking changes (renaming fields, changing field semantics) bump the version.
- Bumps require updating HANDOFF_SCHEMA.md, the orchestrator, and every specialist file in the same PR.

Why this schema is enough (without runtime hooks)

The schema is enforced by: 1. The orchestrator's instructions to ALWAYS emit the outbound block. 2. Every specialist's instructions to refuse delegations missing the block. 3. The orchestrator's instructions to consume the return block before merging work.

This is documentation enforcement — agents follow the rules because their instructions tell them to. It works because: - Subagents are isolated, so a misbehaving agent's damage is bounded - The orchestrator catches return-block violations (missing fields, wrong types) - Refusal is a one-shot rejection: if the specialist returns the refusal template, the orchestrator must re-issue with the missing fields — no work happens until the schema is satisfied

If you later want hard runtime enforcement, add a PreToolUse hook on the `Agent` tool that validates the outbound YAML block before letting the call proceed. That's a one-day project; this framework deliberately doesn't ship hooks because the document-only enforcement covers 95% of the value at 5% of the complexity.

ewpage

05 — Rules and Skills

Two patterns for capturing knowledge that doesn't belong in agents: **rules** (path-globbed invariants) and **skills** (repeatable workflows).

Rules

A rule is a markdown file in `.claude/rules/` with frontmatter declaring which file paths it applies to. When an agent is about to edit a matching file, it must read the rule first.

When to write a rule

Write a rule when: - A bug shipped to production once and must not recur - A pattern is non-obvious from reading the code (e.g. “this column looks like a string but it’s a JSON-encoded string in legacy rows”) - A constraint crosses multiple files but isn’t enforced by types/tests - An anti-pattern is tempting but wrong (e.g. “this looks like dead code but is actually called via reflection”)

Don’t write a rule for: - Things obvious from the code (well-named identifiers do that work) - Style preferences (lint config does that work) - One-off decisions (PR descriptions do that work) - Anything covered by an existing rule (extend the existing one)

Rule file structure

```
---
name: <domain-name>
description: Scoped rules for <domain>. Read before editing any of the path globs below.
applies_to:
  - "<glob1>"
  - "<glob2>"
---

# <Domain Name> Rules

## Hard rules

### <Rule title — short imperative>

**Why:** <one-sentence reason — usually a past incident or invariant>

**How to apply:** <2-4 sentences — what to do, what helper to use, what to check>

### <Next rule>

...

## What NOT to do

- Bullet list of anti-patterns

## How to use this file

1. If you're editing any path in `applies_to`, read this file first.
2. Report which rule files were read in your task summary.

## Cross-links

- `<related orientation map>`
- `<related canonical doc>`
- `<related rule>`
```

Glob patterns

Use standard shell globs. Examples:

```
applies_to:
  - "src/api/**/*.ts"                    # all TypeScript under src/api/
  - "src/db/migrations/**/*.sql"         # all migration SQL files
  - "src/{lib,utils}/auth-*.ts"          # auth-related lib/util files
  - "components/payment/**/*.{tsx,jsx}"  # payment components
  - "ios/Runner/**/*.swift"              # iOS Swift files
  - "lib/screens/**/*.dart"              # Flutter screens
  - "internal/handlers/**/*.go"          # Go API handlers
  - "app/models/*.rb"                    # Rails models
```

Globs are matched against the relative path from project root.

Rule examples by tech

Backend / API

```
### Always validate request body before reading database

**Why:** Three production incidents traced to unvalidated body data flowing to SQL parameters.

**How to apply:** Use `validateRequest(schema)` middleware in `src/lib/validation.ts`. Never r
```

Database / migrations

Migration order is sacred — never reorder, never edit applied migrations

Why: Migrations apply in filename order. Editing an applied migration silently diverges staging

How to apply: New migrations get the next sequential timestamp prefix. Bug fixes to applied mi

Frontend / UI

Never trust component props for security checks — re-validate on the server

Why: A user can edit any prop via React DevTools.

How to apply: Server-side handlers re-validate authorization. Props are for rendering, not auth

Mobile

iOS auto-zooms on input focus when font-size < 16px

Why: iOS Safari WebView feature; jarring UX.

How to apply: Set `*font-size: 16px*` minimum on all `*<input>*`, `*<select>*`, `*<textarea>*`. For v

Infra / DevOps

Never commit secrets — use env vars + secrets manager

Why: Once committed, secrets are in git history forever; rotation required.

How to apply: Add new secrets via `*<your-secrets-manager>*`. The deploy pipeline injects the

Anti-patterns in rules

Long rationale paragraphs. Rules should be ≤ 4 sentences per rule. Move long stories to the canonical doc.

Aspirational rules. “We should always…” — if it’s not enforced, it’s not a rule. Either write it as enforced (with the helper / check) or move to a style guide.

Rules that contradict other rules. Specialist confusion. Have the context-librarian agent reconcile.

Rules with no `applies_to`. Without globs, agents have no signal to read it. Either add globs or move to an orientation map.

Skills

A skill is a directory under `.claude/skills/` containing a `SKILL.md` file. Skills are **repeatable workflows** — task templates that recur frequently enough to deserve a checklist.

When to write a skill

Write a skill when: - The same kind of task arrives 3+ times - The task has multiple steps with verification between them - The cost of skipping a step is high (e.g. forgot to run security review before launching a new auth flow) - A junior team member would benefit from a guided checklist

Don’t write a skill for: - One-off tasks - Tasks where the steps vary too much per instance - Things that are really just a single agent invocation

Skill file structure

```
---
name: <skill-name>
description: <One-sentence description of when to use this skill>
---

# Skill — <Skill Name>

## When to invoke

- Bullet list of trigger conditions

## Workflow

### 1. <Step name>

<2-4 sentences — what to do, what to produce, what to verify>

### 2. <Next step>

...

## Definition of Done

1. <Artifact 1>
2. <Artifact 2>
3. <Verification>

## Cross-links

- `<related agents>`
- `<related rules>`
```

Common skills (most projects benefit from these)

investigate-bug/SKILL.md

Steps: restate bug → identify affected roles → read minimum docs → trace UI → state → API → DB → tests → match touched paths to rules → reproduce → classify severity → propose smallest fix → verify.

build-feature/SKILL.md

Steps: restate feature → identify roles → read minimum docs → run pre-feature checklist (data deltas, RLS, payment impact, etc.) → write acceptance criteria per role → identify rule-covered paths → plan smallest implementation → implement incrementally.

qa-flow/SKILL.md

Steps: identify flows in scope → read testing strategy + relevant orientation map → use browser MCP for UI flows → cover golden path + edge cases + adjacent regressions + cross-role → verify functional + data correctness → severity-rank findings.

compliance-review/SKILL.md

Steps: identify data involved → identify if regulated subjects involved (minors, PHI, EU residents, etc.) → read compliance orientation map + canonical privacy/terms → apply standing checklist (consent, retention, minimization, portability, deletability, processor coverage, disclosure, consent timing, cross-border, marketplace clauses) → flag risks per regulation → produce attorney checklist.

audit-pipeline/SKILL.md

For projects with quality-critical generation pipelines (AI question banks, data ETL, content moderation): trace lifecycle → verify schema → verify required fields on every write → verify validation layers → verify admin review surface → identify gaps → severity-rank findings.

context-refactor/SKILL.md

For maintaining the framework itself: read current INDEX + ledger → categorize suspect content (routing / orientation / rule / skill / agent / outdated / duplicate / conflicting) → move to correct home → consolidate duplicates → flag conflicts (don't silently resolve) → keep root CLAUDE.md small.

Skills vs Rules vs Agents

If the knowledge is…	Put it in…
A path-scoped invariant (“don’t do X when editing Y”)	Rule
A multi-step workflow that recurs	Skill
A persona with its own scope and Definition of Done	Agent
A one-time decision rationale	PR description
Long-form architecture explanation	Canonical doc
Quick orientation for an area	Orientation map (docs/ai-context/<area>.md)

When something feels like it could be 2 of those: prefer rule > skill > agent (in that order). Rules are most precise (path-globbed). Skills are next (named workflow). Agents are heaviest.

Naming and lifecycle

- Rules: lowercase with hyphens, .md suffix. Filename = name.
- Skills: lowercase with hyphens for the directory, SKILL.md inside.
- New rule from a new gotcha: add to existing rule file if domain matches; create new rule file only if it’s a new domain.
- Rule that becomes obsolete (e.g. the underlying constraint was fixed): delete it, don’t leave it as “obsolete.” Stale rules cause confusion.
- Skill that’s invoked < 3x per quarter: consider deleting; the workflow probably wasn’t recurrent enough.

ewpage

06 — Invocation Modes

Three modes coexist by design. Pick by task shape, not by habit.

Mode 1: Default `claude` (no flags)

Use for: - Quick questions about code (“what does X do?”, “where is Y defined?”) - Single-line copy / typo fixes - Pure read tasks (review a file, summarize a function) - One-file bug fixes where the rule + specialist are obvious from the task description - Iterative debugging where you want low-friction back-and-forth

What you get: standard Claude Code main session. Full default tool access (Read, Edit, Write, Bash, all MCP). No maxTurns cap. Can spawn any subagent including built-ins (Explore, Plan, general-purpose) and plugin agents.

How to confirm: startup header shows Claude Code without an agent name.

Mode 2: Explicit orchestrator `claude --agent <project>-orchestrator`

Use for: - Anything spanning more than one role/domain - Anything spanning more than one feature area - Data integrity, payments, security/privacy, compliance work - Bug investigations where root cause may cross layers (UI → API → DB → infra) - Feature builds where acceptance criteria need writing before implementation - Pre-release QA across multiple roles - Anything where the scope is ambiguous

What you get: orchestrator’s full discipline as runtime constraints — tools allowlist (no direct Edit/Write; must delegate), maxTurns cap, Agent(...) restricted to project specialists only. The structured handoff schema is enforced via the orchestrator’s instructions.

How to confirm: startup header shows @<project>-orchestrator. Confirm visually before starting any production-sensitive task.

Mode 3: Specialist mode `claude --agent <specialist-name>`

Also valid for narrow work in a single domain.

Examples: - claude --agent frontend-ui — UI session with browser MCP + Edit/Write - claude --agent backend-api — API work with full data tooling - claude --agent qa-functional — QA pass with browser MCP + test runners - claude --agent legal-compliance — review-only session - claude --agent database — schema/migration work

Specialist sessions inherit the specialist’s tools allowlist and maxTurns cap. They cannot delegate further (subagents do not spawn other subagents per Claude Code

semantics). Use when you know exactly which domain the work lives in and don't need cross-domain coordination.

Why we don't make the orchestrator the default

Setting `agent: "<orchestrator-name>"` in `.claude/settings.json` would force every session into orchestrator mode, which has:

1. **No Edit/Write tools** — every code change requires `Agent(specialist)` delegation. Heavy friction for typos and one-line fixes.
2. **`maxTurns` cap restrictive** for interactive iterative work that often runs 50+ turns.
3. **Body prompt designed as a routing playbook**, not a general-purpose system prompt.
4. **Default Claude Code system prompt is replaced entirely** (per official docs).

The friction cost on everyday work outweighs the marginal enforcement gain. Real always-on enforcement of orchestrator discipline belongs to **hooks** or **`permissions.deny` rules** — both are deferred until you have run 2–3 real tasks through the explicit orchestrator path and confirmed the schema works under load.

Until then: pick the right mode for the task. If you are about to do anything cross-domain or production-sensitive and you are NOT in `--agent <orchestrator-name>`, stop and start a new session with the flag.

Practical recipe

A well-functioning team uses all three modes:

```
# Quick fix (90% of casual work)
cd ~/your-project
claude
# → "Update the button label from 'Sign In' to 'Sign in'"

# Specialist deep-dive (focused domain work)
cd ~/your-project
claude --agent frontend-ui
# → "Audit our Suspense boundaries for client-component leaks"

# Full orchestrated work (multi-domain, production-sensitive)
cd ~/your-project
claude --agent acme-orchestrator
# → "Add a new payment method that requires KYC review and cron-driven activation"
```

Anti-patterns

Using `claude` for everything. Cross-domain work flooded with context. Specialists never invoked. Defeats the point.

Using `claude --agent <orchestrator>` for typos. Forced delegation overhead. Specialist spawn for a one-line fix.

Not knowing which mode you're in. Always check the startup header. If you're not sure, you're probably in the wrong mode.

When you change projects

Each project has its own `.claude/agents/`, so `claude --agent <orchestrator-name>` is project-specific. The orchestrator name should be `<project>-orchestrator` (e.g. `acme-orchestrator`, not just `orchestrator`) so that when you have multiple projects open simultaneously, you can tell at a glance which orchestrator is active.

ewpage

07 — Folder Structure

How to organize documentation so Claude (and humans) always know where to look.

The three tiers

```
docs/
├── ai-context/                ← TIER 1: orientation maps (read by agents)
│   ├── INDEX.md
│   ├── HANDOFF_SCHEMA.md
│   ├── ORCHESTRATION_SPOONFEEDER.md
│   └── <area>-experience.md   ← 50-150 lines per area
│
├── ARCHITECTURE.md            ← TIER 2: canonical references (full detail)
├── API.md
├── TECH_STACK.md
├── <UPPERCASE>.md
│
└── _archive/                  ← TIER 3: frozen historical
    ├── README.md                  ← archive convention
    └── <YYYY-MM>/                 ← dated reports
```

Tier 1 — Orientation maps (docs/ai-context/)

Purpose: Per-area guides Claude reads at the start of any task in that area. Cheaper than reading thousand-line canonical docs.

Format: 50-150 lines per file.

Audience: Agents (primary), humans (secondary).

Naming: <area>-experience.md or <area>-<concern>.md. Examples: - customer-experience.md - admin-experience.md - auth-and-sessions.md - payments.md - realtime-sync.md - data-pipeline.md - infrastructure.md

Body sections: 1. **2-sentence orientation.** What this area is, what it isn't. 2. **Key file paths.** Where to look for the code. 3. **3-5 gotchas worth knowing.** The traps that bit you in production. 4. **Cross-links.** To the canonical docs for full detail and to related rules.

Special files: - INDEX.md — task-type → docs + agent map. The router for the orientation maps. - HANDOFF_SCHEMA.md — the bidirectional handoff schema. - ORCHESTRATION_SPOONFEEDER.md — the human-facing usage guide for the framework. - MIGRATION_LEDGER.md (optional) — tracks doc/structure migrations in progress. - LEGACY_BACKUP.md (optional) — frozen pre-refactor snapshot, if you migrated from a single huge CLAUDE.md.

Tier 2 — Canonical references (docs/<UPPERCASE>.md)

Purpose: Authoritative source of truth on the topics they cover. Full detail. Updated when the underlying truth changes.

Format: As long as needed. No artificial line cap. A canonical doc may be 500-2000 lines.

Audience: Humans (primary), agents (when deeper detail is needed than the orientation map provides).

Naming: UPPERCASE.md. Convention signals "this is canonical."

Common files: - ARCHITECTURE.md — system architecture, components, data flow - API.md — endpoint reference (or link to OpenAPI spec) - TECH_STACK.md — what's installed, why, version constraints - CHANGELOG.md — release history - PRODUCT_REQUIREMENTS.md — PRD - BUSINESS_DOCUMENT.md — non-technical product/business context

Cross-referencing: - Orientation maps in ai-context/ link DOWN to canonical docs. - Canonical docs do NOT link UP to orientation maps (orientation is just a summary; canonical is the truth). - Agents link to either, depending on depth needed.

Tier 3 — Frozen archive (docs/_archive/)

Purpose: Historical material that is no longer authoritative but worth preserving for audits.

Format: Whatever the original was — markdown, HTML, PNG, CSV.

Audience: Audits only. Active docs never link here.

Naming: Date-prefixed subdirectories: <YYYY-MM>/<file>. Examples: - _archive/2026-04/SPRINT_2_GOLIVE_2026-04-25.md - _archive/2026-04/PRODUCTION_HANDOFF_2026-04-26.md - _archive/2025-12/AUDIT_REPORT_2025-12-15.md

Sub-categories under _archive/: - _archive/2026-04/ — dated reports - _archive/copilot-audit/ — old AI tool audits - _archive/screenshots/ — historical UI captures - _archive/seed-history/ — old seed data - _archive/html/ — generated HTML reports

Required: _archive/README.md documenting: - What this directory is - Why it exists

(separation from active docs) - Layout - Three rules: don’t link from active docs, don’t delete, don’t move back to root - “Known link rot” section if you migrated from a structure that had cross-links to now-archived material

What goes where — decision tree

You have a doc to write or move. Where?

Is it a per-area gotcha guide of 50-150 lines?
  → docs/ai-context/<area>.md

Is it the system architecture, API reference, or another full-detail doc?
  → docs/<UPPERCASE>.md

Is it a path-globbed invariant ("don't do X when editing Y")?
  → .claude/rules/<domain>.md

Is it a multi-step workflow that recurs?
  → .claude/skills/<workflow>/SKILL.md

Is it an agent persona definition?
  → .claude/agents/<name>.md

Is it a sprint report, audit, post-mortem, or dated snapshot?
  → docs/_archive/<YYYY-MM>/<file>.md

Is it a one-time decision rationale?
  → PR description or commit message — NOT docs/

Is it just routing ("for X tasks, read Y")?
  → CLAUDE.md or docs/ai-context/INDEX.md

Folder-level conventions

Root level

The root contains ONLY: - Tech-stack config files (package.json, tsconfig.json, Cargo.toml, go.mod, etc.) - Build/deploy config (vercel.json, Dockerfile, docker-compose.yml, etc.) - CI config (.github/, .gitlab-ci.yml) - Test runner config (playwright.config.ts, pytest.ini, etc.) - The framework files (CLAUDE.md, .claude/, docs/) - Application directories (src/, app/, lib/, pkg/, cmd/, etc.) - Static assets (public/, assets/)

The root does NOT contain: - Loose orphan scripts (move to scripts/_archive/ if not actively used) - Screenshots or images (move to docs/_archive/screenshots/ or assets/) - Backup env files (gitignore + delete from history if secrets were committed) - Generated artifacts (gitignore) - Stub directories with one file (consolidate elsewhere)

Tracked-cache cleanup

These directories are commonly tracked by accident and should be gitignored: - Tool MCP caches (.playwright-mcp/, .copilot-audit/) - Test runner per-run state (test-results/, coverage/) - Database CLI temp dirs (supabase/.temp/, firebase/.firebase/) - Build artifacts (.next/, dist/, build/, __pycache__/) - Local logs (logs/)

If they’re already tracked: git rm -r <dir> and add to .gitignore in the same commit.

Env file safety

Add broad gitignore patterns for env file variants:

```
.env
.env.*
!.env.example
.env*.bak*
.env*staging_tmp
.env*tmp
```

If env files with secrets are already in git history, that’s a separate workstream — rotate secrets first, then scrub history with git-filter-repo or BFG. Removing the working-tree copy doesn’t remove the history.

Variants for non-standard projects

Monorepo

```
monorepo-root/
├── CLAUDE.md                  ← root router; mentions per-package CLAUDE.md exists
├── .claude/
│   ├── agents/                ← orchestrator + cross-package specialists
│   ├── rules/                 ← cross-package rules
│   └── skills/
├── docs/                      ← cross-package docs
│   └── ai-context/
└── packages/
    ├── package-a/
    │   ├── CLAUDE.md          ← package-specific router
    │   ├── .claude/agents/    ← package-specific specialists (optional)
    │   └── docs/ai-context/   ← package-specific orientation
    └── package-b/
        └── ...
```

Multi-product in one repo

```
your-product-repo/
├── .claude/agents/
│   ├── orchestrator.md
│   ├── product-a-frontend.md
│   ├── product-a-backend.md
│   ├── product-b-frontend.md
│   └── shared-database.md
├── docs/ai-context/
│   ├── INDEX.md               ← lists tasks per product
│   ├── product-a-experience.md
│   └── product-b-experience.md
```

Mobile + web shared

```
your-app/
├── apps/
│   ├── web/                   ← Next.js / Vite / etc.
│   └── mobile/                ← React Native / Flutter / native
├── packages/                  ← shared libs
└── .claude/agents/
    ├── web-ui.md              ← only edits apps/web/
    ├── mobile-ui.md           ← only edits apps/mobile/
    ├── shared-libs.md         ← only edits packages/
    └── backend-api.md         ← stays separate if hosted elsewhere
```

ewpage

08 — Common Pitfalls

Hard-won lessons from real-world adoption. Read this before bootstrapping a project.

Pitfall 1: Making the orchestrator the project default

Setting `agent: "<orchestrator-name>"` in `.claude/settings.json` SEEMS like the right move (“now everyone gets the discipline always-on!”). It is not.

The orchestrator has: - No Edit/Write tools (delegations required for every change) - A modest `maxTurns` cap (insufficient for interactive iterative work) - A body prompt designed for routing, not general work - A system prompt that REPLACES the default Claude Code system prompt entirely

Daily work (typo fixes, “what does this function do?”, quick prototyping) becomes painful.

Right answer: Don’t set it as default. Document the three invocation modes in your spoonfeeder. Train the team to use `--agent <orchestrator-name>` for production-sensitive cross-domain work and plain `claude` for everything else.

Pitfall 2: Enabling memory on REVIEW-ONLY agents

Per the official Claude Code subagents docs, when you set `memory: project` (or `user` / `local`), the harness automatically grants Read/Write/Edit so the agent can manage its memory files.

This **silently bypasses your `tools:` allowlist** — a `legal-compliance` agent you carefully restricted to Read/Grep/Glob suddenly has Write/Edit because you turned on memory.

Right answer: Never enable `memory` on REVIEW-ONLY agents. For implementation specialists where memory is useful, enable it explicitly and document that the auto-Write/Edit is intentional.

Pitfall 3: Treating documentation enforcement as runtime enforcement

The handoff schema, the universal evidence rule, the failure_condition observation, the soft hop limits — these are **documentation discipline**. They work because agents follow their instructions. They do NOT physically prevent a misbehaving agent from emitting a malformed handoff.

What IS runtime-enforced (by the Claude Code harness): - tools: allowlists - disallowedTools: denylists - maxTurns: caps - Agent(name1, ...) allowlists when orchestrator runs as main thread - mcpServers: per-agent MCP scoping - Subagent context isolation

What is NOT runtime-enforced: - The handoff YAML block being present - The evidence rule being followed - Refusal of vague delegations - Hop limits ("3rd same-specialist delegation → escalate") - "Rules read before edit" - Definition of Done completeness

Right answer: Be honest about which layer enforces what. If you need hard enforcement of a documentation rule, write a PreToolUse hook on the Agent tool that validates the YAML block before letting the call proceed. Don't pretend documentation is enforcement.

Pitfall 4: Agent(name1, name2, ...) only binds in main-thread mode

The Agent(specialist-1, specialist-2, ...) allowlist on the orchestrator's tools field restricts which agents the orchestrator can spawn. **This restriction only takes effect when the orchestrator runs as the main thread** via claude --agent <orchestrator-name>.

When the orchestrator is spawned AS a subagent from another session (via the Agent tool), it cannot spawn further subagents at all (subagents do not spawn subagents per Claude Code semantics) — so the allowlist is moot in that mode.

When the main session reads CLAUDE.md and acts AS-IF the orchestrator (without --agent), the allowlist has no enforcement at all.

Right answer: Document the limitation in the orchestrator file. For hard enforcement on the main thread without --agent, use permissions.deny in .claude/settings.json:

```
{
  "permissions": {
    "deny": [
      "Agent(Explore)",
      "Agent(Plan)",
      "Agent(general-purpose)",
      "Agent(<plugin>:*)"
    ]
  }
}
```

This blocks the main session from spawning built-ins/plugins regardless of which agent it's posing as.

Pitfall 5: MCP allowlist brittleness — list servers, not individual tools

If your specialist needs Chrome DevTools MCP and Playwright MCP, you might be tempted to list every individual tool:

```
tools: Read, Edit, Bash, mcp__chrome-devtools__navigate_page, mcp__chrome-devtools__click,
```

This is brittle. When the MCP server adds a new tool, your specialist won't get it. When tool names change, your specialist breaks silently.

Right answer: Use the documented wildcard syntax (per Claude Code permissions docs):

```
tools: Read, Edit, Bash, mcp__chrome-devtools__*, mcp__playwright__*
```

The wildcard covers all current AND future tools from that MCP server. Other MCP servers (e.g. Supabase MCP, your custom MCP) remain blocked because they're not in the allowlist.

Pitfall 6: Dropping tools: for "convenience"

It's tempting to omit `tools:` on a specialist so it inherits everything from the parent. Don't.

Missing `tools:` means the specialist has full access to: - Every MCP server you've installed (database access, deploy access, third-party APIs) - Every built-in tool

For specialists that legitimately need broad MCP access (e.g. `qa-functional` needs browser MCP), use the wildcard pattern from Pitfall 5. For specialists that don't need MCP, list only the standard tools.

Right answer: Always declare `tools:` explicitly. Defense-in-depth via least privilege.

Pitfall 7: Ignoring `.env*` history

If env files with secrets are already in git history (you committed `.env.local.bak` once, or `.env.staging_tmp` slipped through), removing the file from the working tree doesn't remove the secrets from history.

`git log -p -- .env*` will show every secret that ever leaked. So will any clone of the repo. So will GitHub's commit history.

Right answer: 1. Rotate every credential that was in any leaked env file (treat them as compromised). 2. Use `git-filter-repo` or BFG Repo-Cleaner to scrub the files from history. 3. Force-push the scrubbed history (this is destructive — coordinate with the team). 4. Add broad gitignore patterns to prevent recurrence: `.env*.bak*`, `.env*staging_tmp`, `.env*tmp`.

Pitfall 8: Reorganizing docs without a link sweep

Moving docs is mechanically easy (`git mv`). Verifying links isn't. If you move `docs/AUDIT_2025-12.md` → `docs/_archive/2025-12/AUDIT_2025-12.md` and don't check, you'll discover the broken refs weeks later when someone follows a link in CHANGELOG and gets a 404.

Right answer: Before any doc reorg, capture a baseline of all `docs/*.md` and `.claude/*.md` references:

```
grep -rohE "(docs|\.claude)/[A-Za-z0-9_/-]+\.md" CLAUDE.md docs/ .claude/ | sort -u > /tmp/refs-b
```

After the move:

```
for path in $(cat /tmp/refs-before.txt); do
  [ -f "$path" ] || echo "BROKEN: $path"
done
```

Fix every broken ref before committing the move. Or do the reorg in phases with the sweep between each phase (this is what we did).

Pitfall 9: Treating Claude's training data as authoritative

Claude's training data has a cutoff. Library APIs, SDK signatures, and platform conventions change. Even for things Claude "knows" — verify against current docs before writing code.

Right answer: When writing or refactoring code that touches a library/SDK/platform, instruct the agent to verify against current docs first. Pin doc URLs in the orientation map for that area. Add a rule: "Before writing X-SDK code, fetch the current docs at ."

Pitfall 10: Letting the orchestrator do all the work

If the orchestrator finds itself doing the actual implementation, you've drifted from the pattern. The orchestrator should: - Read enough to delegate well - Issue handoffs - Aggregate returns - Decide if more delegation is needed

The orchestrator should NOT: - Edit code (it has no Edit/Write tool by design) - Run long verifications itself (delegate to qa-functional) - Make architecture decisions without consulting product-flow

Right answer: If the orchestrator is doing implementation, your specialists are too narrow. Either broaden a specialist's scope or add a missing one.

Pitfall 11: Direct push to develop bypassing PR review

Direct pushes feel faster. They skip: - CI / staging deploy verification - Vercel / Cloudflare / Netlify preview URLs - Code review by humans - Auditable PR history

For trivial doc changes the cost of a PR is low. For multi-file reorgs, the cost of a broken merge to develop is high (other engineers' worktrees, broken builds, lost time).

Right answer: Use direct push to develop ONLY when: - The change is < 5 files AND - The change is doc-only OR adds purely-additive code AND - You have explicit authorization in the current session

For everything else: push the branch, open a PR, let the preview deploy run, merge.

Pitfall 12: Not verifying after the work is "done"

A specialist's `status: completed` is the agent's claim, not a guarantee. The orchestrator's job before merging is to verify: - `verified_claims` covers everything in the original `claims` list - `unverified_claims` is empty OR explicitly accepted as future-work - `do_not_pass_downstream_without_verification` items are flagged for the next hop - `tests_run` actually includes the relevant tests - `files_changed` matches `expected_artifacts`

Right answer: Treat the return schema as a contract. If a specialist returns `status: completed` but `verified_claims` is empty, push back: "What did you actually verify?" Specialists should not be allowed to claim done without showing work.

Pitfall 13: Skipping the `failure_condition` field

`failure_condition` feels redundant with `verify_before_acting` — until a specialist mid-task discovers the orchestrator's whole premise was wrong and doesn't know whether to push through or stop.

Without `failure_condition`, specialists tend to push through (sunk-cost on partial work). With `failure_condition`, they have an explicit STOP signal.

Right answer: Make `failure_condition` required in your handoff schema. Reject delegations that omit it. The discipline of articulating "what would prove me wrong" is itself valuable — if the orchestrator can't name a failure condition, the task is too vague.

Pitfall 14: Letting `docs/` rot without a librarian

Without active maintenance, `docs/` accumulates: - Sprint reports from 6 months ago - Audit reports referenced by no one - Outdated architecture decisions - HTML/PNG artifacts - Dated migration plans

After a year, `docs/` has 50+ files and engineers can't tell which are authoritative.

Right answer: Have a `context-librarian` specialist whose job is exactly this. Schedule a quarterly cleanup pass (or more often). Use the `context-refactor` skill to guide the work. Move stale material to `docs/_archive/<YYYY-MM>/`. Never delete (you might need it for an audit).

Pitfall 15: One huge CLAUDE.md instead of a router

If your CLAUDE.md grows past 200 lines, you're using it wrong. CLAUDE.md is the router; detail goes elsewhere.

A 2000-line CLAUDE.md means: - Every Claude Code session loads 2000 lines into context up front - The router and the detail are mixed (you can't tell what's truth vs. what's a passing note) - Updates require touching the same file from many directions - You lose the ability to scope rules to file paths (everything applies everywhere)

Right answer: Refactor a long CLAUDE.md into the framework's tiers. Move per-area gotchas to `.claude/rules/<domain>.md` (with `applies_to` globs). Move orientation to `docs/ai-context/<area>.md`. Move full detail to `docs/<UPPERCASE>.md`. Keep CLAUDE.md as the routing index — golden rules + workflow + tables + cross-links.

We did this on a real codebase. The pre-refactor CLAUDE.md was 2,928 lines. The post-refactor one is ~150. Every existing rule survived; nothing was lost. The routing pattern made it dramatically easier for both Claude and humans to navigate.

Pitfall 16: Bootstrap on a repo with existing Claude Code config

If you ran `/init` previously, or your team has been adding to `CLAUDE.md` for months,

BOOTSTRAP-PROMPT.md must NOT silently overwrite that work.

Right answer: the prompt now includes mandatory pre-flight checks at the top — snapshot to .claude-pre-bootstrap-backup/, naming-collision detection, applies_to: glob conflict detection, drift detection on existing CLAUDE.md, and a decision gate that STOPS if any pre-flight raised a <NEEDS USER CONFIRMATION> flag.

Specific risks: - Existing CLAUDE.md with team rules → Pre-flight 4 (drift detection) flags stale content; Step 9 (merge step) shows a 3-pane diff before writing. - Existing .claude/agents/<name>.md with same name as a proposed specialist → Pre-flight 2 (naming collision check) STOPS for explicit user decision per file. - Existing .claude/rules/ with overlapping applies_to: → Pre-flight 3 detects glob overlap; both rule files would be referenced creating contradictory guidance. - Existing .claude/agents/ with personas that overlap proposed specialists → Pre-flight 5 surfaces the parallel system; user decides migrate vs coexist.

The pre-flight workflow is non-optional — even on apparent greenfield projects, run it. Cost is 30 seconds; benefit is never silently destroying team work.

ewpage

09 — Runbook: Bootstrap a New Project

Step-by-step guide for adopting this framework in a real codebase. Plan for ~2-4 hours of focused work.

Prerequisites

- Claude Code installed (claude --version should print v2.x or later)
- A project repository (any tech stack)
- Read access to this framework at ~/Desktop/claude-orchestration-framework/
- 2-4 hours of focused time (don't rush this; the foundation lasts years)

Phase 0 — Decide if you need this framework (10 min)

Skip this framework if: - Your project is a solo prototype - Your project has fewer than 4 distinct domain areas (frontend, backend, etc.) - Your team uses Claude Code for the occasional question, not as a development partner - Your project is < 5,000 lines of code

Use this framework if: - Your project has multiple roles (admin, customer, partner, etc.) - Multiple data systems are in play (DB + cache + search + queue) - Real compliance/security/payments concerns exist - A team uses Claude Code daily on the same codebase - You've experienced cascading hallucinations or context flooding

Phase 1 — Inventory your project (30 min)

Open Claude Code in your project root:

```
cd ~/your-project
claude
```

Paste prompts/INVENTORY-PROMPT.md (from this framework). Claude will: - Scan your codebase structure - Propose a list of specialists based on your domain boundaries - Surface clutter that should be archived - Identify your protected branches (main, develop, etc.) - Identify your tech stack and which MCP servers will be useful

You answer: confirm/adjust the proposed specialist list. This is the foundation of everything that follows.

Phase 2 — Bootstrap the framework files (45 min)

In the same Claude Code session, paste prompts/BOOTSTRAP-PROMPT.md. Reference this framework's path so Claude can read templates:

```
The framework lives at /Users/<you>/Desktop/claude-orchestration-framework/.
Read templates from there. Customize for this project: <project-name>.
```

Claude will: 1. Create .claude/agents/<project>-orchestrator.md from templates/orchestrator-agent.md.template 2. Create one .claude/agents/<specialist>.md per specialist, from the appropriate template 3. Create docs/ai-context/HANDOFF_SCHEMA.md from the template 4. Create docs/ai-context/INDEX.md with task → docs map 5. Create docs/ai-context/ORCHESTRATION_SPOONFEEDER.md with usage guide 6. Create skeleton docs/ai-context/<area>-experience.md files (one per major domain) 7. Create CLAUDE.md

at root (the router) 8. Create docs/_archive/README.md 9. Update .gitignore with framework-relevant entries

You verify each file before saving. Claude shouldn't ship anything you haven't read.

Phase 3 — Add your first rules (30 min)

The hardest part of this framework is writing useful rules. They take real domain knowledge.

In the Claude Code session, ask:

```
Based on the codebase scan, propose 3-5 rule files for .claude/rules/.
For each, list the path globs (applies_to:) and 2-3 hard rules with WHY + HOW TO APPLY.
Do not create any rule that's just style preference — only invariants where breaking them caused
or could cause a real production issue.
Wait for my approval before creating any files.
```

Review each proposal. Approve, edit, or reject. Common rule files to start with: - backend-api.md (or your equivalent server-side concern) - frontend-ui.md (or mobile-ui.md) - database.md - auth-security.md - testing.md

Aim for 3-5 rules per file at first. You'll add more over time as production teaches you new gotchas.

Phase 4 — Add 1-2 starter skills (20 min)

Most projects benefit from at least these skills: - investigate-bug/SKILL.md - build-feature/SKILL.md

Ask Claude:

```
Create .claude/skills/investigate-bug/SKILL.md and .claude/skills/build-feature/SKILL.md based
on templates/skill.md.template, customized for this project's tech stack and roles.
```

You can add more skills later as patterns recur (qa-flow, compliance-review, audit-pipeline, context-refactor).

Phase 5 — Verify (15 min)

Run these checks:

```
# All agents register
claude agents

# Doc-link sweep — find broken refs
grep -rohE "(docs|\.claude)/[A-Za-z0-9_/-]+\.md" CLAUDE.md docs/ .claude/ | sort -u | while read p
```

```
# Build still passes (whatever your build command is)
npm run build  # or: pytest, go build, cargo build, etc.
```

Fix any breakage before moving on. Common issues: - Agent file YAML invalid (claude agents will tell you) - Markdown link in CLAUDE.md or INDEX.md to a file that doesn't exist (typo) - Rule glob points to a path that doesn't match anything (mostly harmless but confusing)

Phase 6 — Run a real task through the orchestrator (30 min)

Pick a real task — ideally a small bug or a small feature. Open a NEW Claude Code session in orchestrator mode:

```
cd ~/your-project
claude --agent <project>-orchestrator
```

Verify the startup header shows @<project>-orchestrator.

Give it a real task. Watch what happens: - Does the orchestrator emit the outbound handoff YAML block? - Do specialists return the inbound YAML block? - Does the orchestrator catch any rejected_claims correctly? - Does verification (tests, build) actually run?

If anything is off: - Tighten the orchestrator's "outbound discipline" section - Tighten the specialist's "incoming handoff validation" section - Add a missing rule - Add a missing orientation map

Don't add hooks yet. Get the documentation enforcement working through 2-3 real tasks first.

Phase 7 — Document for your team (20 min)

Add to your project's existing onboarding docs: - "We use Claude Code with a custom orchestration setup. See docs/ai-context/ORCHESTRATION_SPOONFEEDER.md." - "Always pick the right invocation mode: `claude` for quick work, `claude --agent <orchestrator>` for cross-domain work." - "Don't push to main without project-owner approval."

Optional: announce in your team chat with a 30-second demo of the orchestrator handling a multi-step task.

Phase 8 — Add hardening as needed (optional, after 2 weeks)

After running real tasks for a couple weeks, decide if you need:

Hop limits via PreToolUse hook

If you see the orchestrator delegating to the same specialist 3+ times in a row without progress, add a hook that counts Agent invocations per turn and warns past N hops.

Hard schema enforcement via PreToolUse hook

If specialists keep emitting malformed return blocks, add a hook that validates the YAML before letting the call complete.

`permissions.deny` for built-ins/plugins on main thread

If team members keep accidentally invoking Explore/Plan/plugin agents from default `claude` sessions when they should be in orchestrator mode, add to `.claude/settings.json`:

```
{
  "permissions": {
    "deny": [
      "Agent(Explore)",
      "Agent(Plan)",
      "Agent(general-purpose)",
      "Agent(<plugin>:*)"
    ]
  }
}
```

Per-agent memory for cross-session learning

For implementation specialists where institutional knowledge accumulates (debugging recipes, schema gotchas), add `memory: project` to the agent frontmatter. **Do NOT enable for REVIEW-ONLY agents** — see Pitfall 2.

Phase 9 — Quarterly maintenance

Every 3 months, dedicate a session to: 1. Run the `context-refactor` skill (or just have the `context-librarian` specialist do a sweep) 2. Move stale dated reports to `docs/_archive/<YYYY-MM>/` 3. Reconcile any rule conflicts 4. Update orientation maps for areas where reality has drifted 5. Verify all agents still register after dependency updates

Common time-sinks to avoid

- **Don't try to make every rule perfect on day 1.** Ship 5 rules; add 1 per month as production teaches you.
- **Don't over-specialize.** 6-8 specialists is plenty. Splitting too narrowly creates routing confusion.
- **Don't write canonical docs from scratch in this framework.** Use what you already have. The framework adds the orientation/router layer on top.
- **Don't manually merge agent files when adding a rule.** Always edit the rule file; agents reference it via `applies_to`.

When to stop and ask

- If `claude agents` shows fewer agents than you expect → YAML frontmatter syntax error
- If specialist outputs don't include the return YAML block → the specialist's body

needs the “Return schema” section
- If the orchestrator keeps delegating in a loop → check failure_condition is articulated
- If a specialist refuses a delegation → check the orchestrator’s outbound block has all required fields

End state

You have: - A <project>-orchestrator and 4-10 specialists - 3-5 rule files covering your highest-risk paths - 2-4 skills for recurring workflows - Three-tier docs (orientation maps + canonical refs + archive) - A team that knows how to pick invocation modes - Auditable handoffs for every cross-domain task

Time invested: 2-4 hours setup + ~30 min/quarter maintenance.

Time saved: every cascading-hallucination incident you DIDN’T have. Every cross-domain task that got the right specialists without you thinking about it. Every new engineer who could navigate the codebase without 2 weeks of pairing.

ewpage

Part II — Prompts

The three prompts to paste into Claude Code at different points in the lifecycle.

ewpage

INVENTORY PROMPT

Paste this into Claude Code at the root of the project you want to bootstrap. Adjust <framework path> to match your install location.

I want to adopt the Claude Orchestration Framework on this project. Before we generate any files, do a read-only inventory.

The framework lives at <framework path> (default: /Users/<you>/Desktop/claude-orchestration-framework/). Read docs/01-PRINCIPLES.md, docs/02-ARCHITECTURE.md, and docs/03-AGENTS-GUIDE.md from there before answering. Do not modify any files in this project yet.

⚠ Two universal rules for this entire pass

1. **Evidence-first.** Every claim you make must cite a real file path, command output, or filename. The phrase “I infer” or “appears to be” without a specific file:line citation is not acceptable.
2. **Ask, don’t guess.** If a fact cannot be confirmed by reading a specific file or running a specific command, mark it <NEEDS USER CONFIRMATION: <one-line question for me>> and EXPLAIN what you tried before giving up. Examples:
   - <NEEDS USER CONFIRMATION: Is this project named "acme" or "acme-store"? package.json says "@acme/store-monorepo" — slug is ambiguous.>
   - <NEEDS USER CONFIRMATION: Are EU users in scope? No GDPR-related env vars or consent UI found, but README mentions "international expansion".>

Surface ambiguity. Never invent a fact to fill a gap.

Discovery commands — run these in order

Before writing anything, execute this discovery sequence and capture the results in your context. If a command/file is not present, note it as missing and move on.

Step A — Top-level inventory

```
pwd
ls -la
find . -maxdepth 1 -type f | sort
find . -maxdepth 2 -type d | sort
git remote -v
git branch -a
git log --oneline -5
```

Step B — Project name + tech stack manifest discovery

Read whichever of these exist (in this order); stop after the first one found that has a clear name field:

File	Looks for
package.json	name, description, dependencies, scripts (Node/JS/TS)
pyproject.toml	[project] name, dependencies (Python)
setup.py	name=, install_requires= (legacy Python)
Cargo.toml	[package] name, [dependencies] (Rust)
go.mod	module <path>, require (Go)
Gemfile + *.gemspec	gem name, deps (Ruby)
composer.json	name, require (PHP)
pubspec.yaml	name:, dependencies: (Flutter/Dart)
*.podspec / Podfile	iOS native dependencies
build.gradle / build.gradle.kts	Android / Kotlin
*.csproj	.NET project name
package.swift	Swift Package Manager
mix.exs	Elixir

If MULTIPLE manifests exist (monorepo), list all of them and ask: <NEEDS USER CONFIRMATION: Which manifest is the canonical project name source?>

If NONE exist: <NEEDS USER CONFIRMATION: No standard manifest found. Should I use the directory name "<dirname>" as the project name?>

Step C — Framework + service detection

Read these files if present (don't skip — each tells Claude something specific):

File	Tells you
next.config.js / next.config.ts	Next.js (web framework)
nuxt.config.ts	Nuxt (Vue web framework)
vite.config.ts / vite.config.js	Vite (build tool — usually Vue/React/Svelte)
svelte.config.js	SvelteKit
astro.config.mjs	Astro
remix.config.js	Remix
gatsby-config.js	Gatsby
angular.json	Angular
vue.config.js	Vue CLI
app.json / metro.config.js	React Native / Expo
pubspec.yaml (lib: flutter)	Flutter
tsconfig.json	TypeScript target/strict mode
prisma/schema.prisma	Prisma ORM (database)
drizzle.config.ts	Drizzle ORM (database)
knexfile.js	Knex (DB query builder)
*.sql files in migrations/ or db/	Raw SQL migrations
Dockerfile / docker-compose.yml	Container setup
vercel.json	Vercel deploy
netlify.toml	Netlify deploy
wrangler.toml	Cloudflare Workers
serverless.yml	Serverless framework
fly.toml	Fly.io deploy
app.yaml	Google App Engine
Procfile	Heroku / similar PaaS
firebase.json	Firebase
amplify.yml / amplify/	AWS Amplify
terraform/*.tf / cdk.json	IaC (Terraform / AWS CDK)
.github/workflows/*.yml	GitHub Actions CI/CD
.gitlab-ci.yml	GitLab CI
.circleci/config.yml	CircleCI
playwright.config.ts	Playwright E2E tests
cypress.config.ts	Cypress E2E tests
vitest.config.ts / jest.config.js	Unit test runner
pytest.ini / tox.ini	Python test config

Step D — External service detection (.env / dependencies)

Read .env.example (or .env.sample / .env.template — whichever exists). For each environment variable, infer the service:

Env var pattern	External service
STRIPE_*, STRIPE_SECRET_KEY	Stripe (payments)
RESEND_*, SENDGRID_*, MAILGUN_*, POSTMARK_*	Transactional email
TWILIO_*	Twilio (SMS / voice)
OPENAI_API_KEY, ANTHROPIC_API_KEY, GEMINI_*	LLM provider
SUPABASE_*, NEXT_PUBLIC_SUPABASE_*	Supabase (backend)
FIREBASE_*	Firebase
CLERK_*, NEXTAUTH_*, AUTH0_*, WORKOS_*	Auth provider
POSTHOG_*, MIXPANEL_*, SEGMENT_*, AMPLITUDE_*	Analytics
SENTRY_*	Error monitoring
DATADOG_*, NEW_RELIC_*	APM / observability
CLOUDFLARE_*, R2_*, S3_*, GCS_*	Storage / CDN
ALGOLIA_*, MEILISEARCH_*, ELASTICSEARCH_*	Search
REDIS_URL, UPSTASH_*	Cache / queue
DAILY_*, LIVEKIT_*, AGORA_*, PUSHER_*, ABLY_*	Real-time / video
STRIPE_* (multiple)	Subscription billing or marketplace

Cross-check by reading the dependencies section of the manifest discovered in Step B. The two should agree.

Step E — Source-tree shape

```
find . -maxdepth 4 -type d \
  -not -path './node_modules/*' \
  -not -path './.git/*' \
  -not -path './.next/*' \
  -not -path './dist/*' \
  -not -path './build/*' \
  -not -path './target/*' \
  -not -path './venv/*' \
  -not -path './__pycache__/*' \
  | sort | head -100
```

Then for the apparent source root (src/, app/, lib/, internal/, cmd/, etc.):

```
find <source-root> -maxdepth 3 -type d | sort
ls <source-root>/<api-or-server-dir> 2>/dev/null | head
ls <source-root>/<components-or-ui-dir> 2>/dev/null | head
```

Step F — Documentation discovery

```
find docs -type f -name "*.md" 2>/dev/null | sort
ls docs/ai-context/ 2>/dev/null
find . -maxdepth 1 -type f -name "*.md" | sort   # README, CONTRIBUTING, etc.
```

For each doc found, read its first 20 lines to categorize (canonical / orientation / archive / workflow).

Step G — Hygiene scan

```
ls .copilot-audit .playwright-mcp test-results logs supabase/.temp 2>/dev/null
find . -maxdepth 1 -type f -name "_*" -o -name "check-*" -o -name "test-*" 2>/dev/null
find . -maxdepth 1 -type f \( -name "*.png" -o -name "*.jpg" -o -name "*.html" \) 2>/dev/null
find . -maxdepth 2 -type f -name ".env*" 2>/dev/null
git ls-files | grep -E "(\\.env|secret|credential)" | head
```

Step H — Branch and remote check

```
git branch -a | head -20
git remote -v
git log --all --pretty=format:"%h %s" --since="3 months ago" --grep="protected\|deploy\|main\|devel
cat .github/workflows/*.yml 2>/dev/null | grep -E "branches:|push:" | head
```

This identifies protected branches by what CI runs against.

Now produce the structured inventory in 7

sections

Use the discovery results above. Cite SPECIFIC files for every claim.

1. Project identity

- Project name (one phrase) — cite which manifest you read it from
- Project slug (lowercase-hyphenated) — derived from the name; if ambiguous, ASK
- One-sentence description — from manifest description or README first paragraph
- Primary tech stack — list each component with the file that confirmed it
  - Language: <language> (from <file>)
  - Framework: <framework + version> (from <file>)
  - Database: <db> (from <file> or .env.example)
  - Auth: <provider> (from deps or env vars)
  - Payments: <provider or "none"> (from deps or env vars)
  - Email: <provider or "none">
  - Analytics: <provider or "none">
  - Real-time: <provider or "none">
  - LLM: <provider or "none">
  - Other notable services: <list>
- Deployment target — cite the deploy config file
- CI/CD — cite the workflow file(s)
- Protected branches — list with confirmation source
- Test runners — unit + E2E (cite config files)

2. Roles and audiences

List every distinct user type the project serves. Infer from: - Route group structure (/admin, /(auth), /(customer), etc.) - Permission-model files (RBAC config, middleware) - README or PRD if present

For each role: - Name - One-sentence description of what they do - Evidence (file path or doc that confirms)

If unsure whether a role exists: <NEEDS USER CONFIRMATION: Is "<proposed-role>" a real user type, or is it a code abstraction?>

3. Domain boundaries (proposed specialists)

Propose specialists. For each: - name: (lowercase-hyphenated, domain-shaped not technology-shaped) - One-sentence description of what it owns - Whether it should be REVIEW-ONLY or implementation - Path globs it would primarily edit (must match real directories — verify with ls) - Evidence: which signals from Steps C/D/E led you to propose this specialist

Aim for 5-10 specialists. Always include: - An orchestrator named <project-slug>-orchestrator - A qa-functional if there's any user-facing surface (test config exists in Step C) - A release-devops if there's any deploy automation (deploy config + CI exist) - A security-privacy (REVIEW-ONLY) if there's any user data - A legal-compliance (REVIEW-ONLY) ONLY if regulatory exposure is confirmed (children's data, health, finance, EU users) — NOT speculatively. If unsure: <NEEDS USER CONFIRMATION: Is this project subject to <regulation>? I see <evidence or lack thereof>.>

For each specialist that DOESN'T have obvious need-to-exist evidence in the codebase, MARK it with <NEEDS USER CONFIRMATION> rather than including it speculatively.

4. Path-globbed rules (proposed)

For each specialist's domain, propose 1-2 rule files. For each rule file: - File name (<.claude/rules/<name>.md>) - applies_to: glob list (verify globs match real files via find or ls) - Top 3-5 hard rules — each WITH source-confirmed evidence (cite file:line where the gotcha CURRENTLY lives in code or where a recent commit/PR fixed it)

If you cannot find evidence for a proposed rule: - Either: don't propose it - Or: mark <NEEDS USER CONFIRMATION: I think rule X applies because Y, but I can't find a code example. Is this rule real?>

NEVER include rules that are just generic best practices. The framework's value is project-specific gotchas, not lint config.

5. Existing documentation tier

Categorize every file in docs/ (or wherever the project keeps docs): - **Canonical** — full-detail, authoritative, currently accurate. Stays at docs/<UPPERCASE>.md. - **Orientation candidate** — domain-specific, would benefit from being condensed into docs/ai-context/<area>.md. Suggest the new filename. - **Archive candidate** — dated reports, sprint snapshots, post-mortems. Suggests the archive subdir (docs/_archive/<YYYY-MM>/). - **Active workflow** — currently-in-progress plans. Keep but flag for re-categorization.

Cite each file's location and your categorization rationale.

6. Clutter / hygiene findings

Surface (don't fix yet) — for each, cite the path and the specific concern: - Tracked tool caches (.playwright-mcp/, .copilot-audit/, test-results/, build artifacts) — list each with file count - Orphan scripts at root level — list each with one-line description of what it appears to do - Empty stub directories — list - Loose screenshots / images at root — list - Env file backups (.env.local.bak*, .env.staging_tmp, etc.) — **flag CRITICAL if tracked in git** (potential secrets in history) - Files with stale paths (e.g. VSCode launch.json pointing to a renamed project) — list with diff of expected vs actual path - Any obvious security concerns: - Env files in git history (git log -p -- .env* returns content) - Hardcoded secrets in source (grep for sk_live_, password=, common patterns) - Bind addresses set to 0.0.0.0 in non-server code

7. Invocation guidance for the team

Propose a one-paragraph snippet for the team's onboarding doc explaining when to use: - claude (default — no flags) - claude --agent <project-slug>-orchestrator (cross-domain / production-sensitive) - claude --agent <specialist> (narrow single-domain)

Customize for THIS project's actual specialist list and protected-branch names.

Output format rules

- Use the section headings above EXACTLY. Number them 1-7.
- Cite a specific file path or command output for every claim. "I see" without a citation is not enough.
- For anything you can't verify, mark <NEEDS USER CONFIRMATION: <specific question>> — do NOT guess silently.
- Do NOT create any files in this pass. This is read-only investigation.
- After producing the inventory, end with a section called ## Open questions listing every <NEEDS USER CONFIRMATION> from above as a numbered list — so I can answer them in one pass.
- Then ask which sections I want to adjust before bootstrapping.

What success looks like

After this prompt, I should be able to: 1. Read your inventory and immediately know what specialists you propose, with rationale per specialist 2. Answer all open questions in one message 3. Give you the green light (or adjustments) before any file is generated

If your output is missing any of: discovery command results, evidence citations per claim, or an Open Questions section — start over. The discipline of this prompt is the whole point.

(End of prompt.)

ewpage

BOOTSTRAP PROMPT

Paste this AFTER you have completed the INVENTORY-PROMPT pass and approved the proposed specialists. Adjust <framework path> to match your install location.

I've reviewed the inventory. Now bootstrap the Claude Orchestration Framework on this project. Use the templates from <framework path> (default: /Users/<you>/Desktop/claude-orchestration-framework/templates/). Read those template files before generating anything.

⚠ PRE-FLIGHT SAFETY CHECKS (run BEFORE creating anything)

This project may already have some Claude Code configuration (e.g. from running /init, or from prior team work). Bootstrap is designed to coexist with existing config, but ONLY if you detect collisions first. Run these checks in order. Do NOT skip even if the inventory said the project was greenfield.

Pre-flight 1 — Auto-snapshot (mandatory)

Before any write, create a backup of any existing Claude Code config:

```
mkdir -p .claude-pre-bootstrap-backup
[ -f CLAUDE.md ] && cp CLAUDE.md .claude-pre-bootstrap-backup/
[ -d .claude/agents ] && cp -r .claude/agents .claude-pre-bootstrap-backup/
[ -d .claude/rules ] && cp -r .claude/rules .claude-pre-bootstrap-backup/
[ -d .claude/skills ] && cp -r .claude/skills .claude-pre-bootstrap-backup/
[ -f .claude/settings.json ] && cp .claude/settings.json .claude-pre-bootstrap-backup/
[ -f .claude/settings.local.json ] && cp .claude/settings.local.json .claude-pre-bootstrap-backup/
[ -d docs/ai-context ] && cp -r docs/ai-context .claude-pre-bootstrap-backup/
ls -la .claude-pre-bootstrap-backup/
```

Report what was backed up. If the backup directory is empty, the project had no prior Claude Code config — proceed without further pre-flight concerns.

If the backup directory has content, continue with pre-flight 2-5.

Pre-flight 2 — Naming collision check (per file)

For EACH file you plan to create, check if the destination path already exists:

```
# For each proposed agent
for agent in <project-slug>-orchestrator <specialist-1> <specialist-2> <...>; do
  if [ -f ".claude/agents/$agent.md" ]; then
    echo "COLLISION: .claude/agents/$agent.md already exists"
  fi
done

# For each proposed rule
for rule in <name-1> <name-2> <...>; do
  if [ -f ".claude/rules/$rule.md" ]; then
    echo "COLLISION: .claude/rules/$rule.md already exists"
  fi
done

# For each proposed skill
for skill in investigate-bug build-feature <...>; do
  if [ -d ".claude/skills/$skill" ]; then
    echo "COLLISION: .claude/skills/$skill/ already exists"
  fi
done
```

For EACH collision found, STOP and ASK: > <NEEDS USER CONFIRMATION: .claude/agents/<name>.md already exists. Options: (a) overwrite (existing content goes to backup), (b) skip this agent, (c) merge content. Which?>

Do NOT silently overwrite ANY collision. If the user picks (c) merge, propose the merged content and get explicit approval before writing.

Pre-flight 3 — applies_to: glob conflict check

For each new rule file you plan to create, check if existing rule files have an OVERLAPPING applies_to: glob:

```
# Read all existing rule file frontmatters
for f in .claude/rules/*.md; do
  [ -f "$f" ] && echo "=== $f ===" && awk '/^applies_to:/,/^---$/' "$f" | head -10
done
```

If your proposed applies_to: glob (e.g. "src/api/**/*.ts") overlaps with an existing rule's glob, BOTH rule files will be referenced when editing matching files — leading to potentially contradictory rules. STOP and ASK: > <NEEDS USER CONFIRMATION: My proposed .claude/rules/api.md (applies_to: "src/api/**/*.ts") overlaps with existing .claude/rules/server.md (applies_to: "src/api/**,src/server/**"). Options: (a) merge into the existing file, (b) tighten my proposed glob to a non-overlapping subset, (c) accept the overlap (both will be referenced). Which?>

Pre-flight 4 — Drift detection on existing CLAUDE.md

If CLAUDE.md exists, READ IT and compare against current project reality:

```
# Re-read current tech stack from manifest
cat package.json 2>/dev/null | head -30
cat Cargo.toml 2>/dev/null | head -30
cat go.mod 2>/dev/null | head -10
cat pyproject.toml 2>/dev/null | head -30

# Compare to what existing CLAUDE.md says
cat CLAUDE.md
```

Look for stale claims in the existing file: - Tech stack mentions that don't match current dependencies (e.g. file says "Vue 3" but package.json has React) - Directory

paths that don't exist anymore (e.g. file says src/components/ but the project moved to app/components/) - Conventions referencing removed features (e.g. mentions of /api/v1/legacy/ routes that were deleted) - Branch names that no longer exist (e.g. mentions master but the repo is main)

For EACH stale entry found, STOP and ASK: > <NEEDS USER CONFIRMATION: Existing CLAUDE.md says "<stale claim>" but current reality is "<actual>". Should I drop / update / preserve as-is?>

Do NOT silently drop content. The existing rules may be valuable context the team added intentionally.

Pre-flight 5 — Existing agents differ in style

If .claude/agents/ already has files, check whether they follow a similar persona/contract style. Two parallel conventions in the same repo confuses the team.

```
ls .claude/agents/ 2>/dev/null
# For each existing agent, read its frontmatter + first ~30 lines of body
for f in .claude/agents/*.md; do
  [ -f "$f" ] && echo "=== $f ===" && head -40 "$f"
done
```

If existing agents exist with persona contracts that overlap with what you'd create, STOP and ASK: > <NEEDS USER CONFIRMATION: Existing agent .claude/agents/code-reviewer.md exists with its own persona. Should the new <project-slug>-orchestrator coordinate WITH it (treat it as another specialist), REPLACE it, or IGNORE it? If you have invested in the existing agent system, replacement risks losing team knowledge.>

Also check whether existing agents use the new tools: / maxTurns: / Agent(...) allowlist patterns — if not, propose adding them as part of bootstrap (with user approval per agent).

Decision gate — STOP if ANY pre-flight check raised an issue

If pre-flight 1-5 raised even one <NEEDS USER CONFIRMATION> flag, STOP and present ALL flags as a numbered list. Wait for the user to answer EVERY flag before proceeding to Step 1 below. Do not partially proceed.

If pre-flight 1-5 raised zero issues (truly greenfield Claude Code setup), proceed to Step 1.

What to create (order matters)

Step 1 — Create the directory skeleton

```
.claude/agents/
.claude/rules/
.claude/skills/
docs/ai-context/
docs/_archive/
```

Step 2 — Create docs/ai-context/HANDOFF_SCHEMA.md

Use templates/HANDOFF_SCHEMA.md.template. Substitute <orchestrator-name> with <project-slug>-orchestrator. Customize the worked examples to use realistic file paths from THIS project's codebase (not generic placeholders). Show me the file before saving.

Step 3 — Create docs/_archive/README.md

Use templates/archive-README.md.template. No placeholder substitution needed. Show me the file before saving.

Step 4 — Create docs/ai-context/INDEX.md

Use templates/INDEX.md.template. Populate the routing table from the inventory's section 3 (specialists) and section 5 (orientation candidates). For each row: task type → which orientation map(s) to read → which rules might apply → which agent owns it. Show me the file before saving.

Step 5 — Create the orchestrator agent

.claude/agents/<project-slug>-orchestrator.md — use templates/orchestrator-agent.md.template. Substitute: - <PROJECT_NAME> — the project's identity from inventory section 1 - <PROJECT_SLUG> — the lowercase-hyphenated slug - <SPECIALIST_LIST> — comma-separated list of specialist names from inventory section 3 -

<PROJECT_SPECIFIC_ANTI_PATTERNS> — 3-5 anti-patterns specific to this project (derived from inventory)

Show me the file before saving.

Step 6 — Create each specialist agent

For each specialist from the inventory section 3: - If REVIEW-ONLY: use templates/review-only-agent.md.template - If implementation: use templates/specialist-agent.md.template

Substitute placeholders. The Incoming handoff validation and Return schema (required) sections are IDENTICAL across every specialist — paste verbatim from the template. Customize the specialist-specific sections (When to use / When NOT to use / Required reading / I CAN / I CANNOT / Definition of Done / Cross-links).

Show me each file before saving. Do them in this order so I can check the pattern, then approve subsequent ones in batch: 1. First specialist: full review 2. Second specialist: full review 3. Remaining specialists: batch review (just confirm correct frontmatter + correct cross-links)

Step 7 — Create skeleton orientation maps

For each major domain identified in inventory section 5, create docs/ai-context/<area>.md with a 50-150 line skeleton. Format: - 2-sentence orientation - "Key file paths" section (extracted from the codebase scan) - "Top gotchas" section (start with placeholders, mark <TODO: fill from real bugs>) - "Cross-links" section (link to canonical docs that already exist)

Show me each one before saving.

Step 8 — Create docs/ai-context/ORCHESTRATION_SPOONFEEDER.md

Use templates/SPOONFEEDER.md.template. Customize the invocation modes section with the project's specific orchestrator name and specialist list.

Step 9 — Create / update the project's CLAUDE.md router

This file goes at the repo root. Should be UNDER 200 lines.

If CLAUDE.md doesn't exist: create it. Include: - 1-paragraph project identity - Golden rules (always include "never push to protected branches without owner approval") - Mandatory workflow (numbered steps) - Routing tables: task touches X → read Y; task class → use agent Z - Skills list (cross-link to .claude/skills/) - Reference docs list (cross-link to docs/<UPPERCASE>.md) - Branches & deploy (target deploy URLs per branch) - Local dev commands

If CLAUDE.md EXISTS (verified during pre-flight 1):

1. The original is already snapshotted in .claude-pre-bootstrap-backup/CLAUDE.md.
2. **Read the existing file in full.** Identify which sections are: (a) project-specific rules the team added intentionally, (b) auto-generated /init content that may be stale, (c) generic best-practice content the framework's template covers anyway.
3. Produce a PROPOSED MERGED version that:
   - Preserves ALL section (a) content (project-specific team rules)
   - Drops or updates section (b) content if pre-flight 4 (drift detection) flagged it as stale
   - Replaces section (c) content with the framework's structured router
4. **Show the user a 3-pane diff** before writing:
   - LEFT: existing file (from backup)
   - RIGHT: proposed merged file
   - MIDDLE: a per-section disposition list ("preserved", "updated for drift", "replaced by template router", "dropped — covered by template")
5. Wait for explicit user approval ("yes, write the merged file") before writing. If the user pushes back on any section, revise and re-show the diff.

⚠ **Keep this file UNDER 200 lines** post-merge. CLAUDE.md is auto-loaded into every Claude Code session — it's a tight router, not a manual.

If the merged file would exceed 200 lines: propose moving sections to .claude/rules/<NAME>.md (path-globbed) or docs/ai-context/<area>.md (orientation map) instead of bloating the router. Get user approval per moved section.

Use templates conceptually but adapt to this project's actual structure.

Step 10 — Update .gitignore

Append (don't overwrite) these entries to the existing .gitignore:

```
# Bootstrap backup — keep local-only, do not commit
.claude-pre-bootstrap-backup/

# Tool caches that should never be committed
.copilot-audit/
.playwright-mcp/
test-results/
logs/

# Vercel/CLI env exports — broader pattern
.env*.bak*
.env*staging_tmp
.env*tmp

# Claude Code worktrees (local-only)
.claude/worktrees/
.claude/launch.json
```

The .claude-pre-bootstrap-backup/ line is critical — that directory contains the original Claude Code config from BEFORE the bootstrap and should NEVER be committed.

If the project uses Supabase, also add:

```
supabase/.temp/
```

If the project uses Firebase:

```
.firebase/
firebase-debug.log
```

(Skip whichever doesn't apply.)

Constraints

- **Do not commit anything.** Show me each file. I'll commit at the end.
- **Do not modify existing application code.** Only create new files in .claude/, docs/ai-context/, docs/_archive/, and update .gitignore + CLAUDE.md.
- **Pre-flight checks are mandatory** (see top of prompt). Do not skip even on apparent greenfield. If pre-flight raises ANY <NEEDS USER CONFIRMATION> flag, STOP and present all flags before any file write.
- **Snapshot first, write second.** Pre-flight 1 creates .claude-pre-bootstrap-backup/ — that backup is your safety net. If you have NOT created the backup, do not proceed to Step 1.
- **Never silently overwrite.** Any pre-existing file at a destination path you'd write requires explicit user approval (per pre-flight 2). "Merge" is not the default — ASK whether to overwrite, skip, or merge.
- **Cite evidence for every gotcha.** If you can't find evidence, mark <NEEDS USER CONFIRMATION> rather than guessing.
- **Use the wildcard MCP pattern** for browser-testing specialists: mcp__chrome-devtools__*, mcp__playwright__*, etc. (per Claude Code permissions docs). Do NOT enumerate individual MCP tools.
- **REVIEW-ONLY specialists must NOT have** **memory:** in frontmatter (per Pitfall 2 in docs/08-COMMON-PITFALLS.md).

After all files are created — verify

Run these commands and report results:

```
# 1. All agents register
claude agents

# 2. Doc-link sweep — find broken refs
grep -rohE "(docs|\.claude)/[A-Za-z0-9_/-]+\.md" CLAUDE.md docs/ai-context/ .claude/agents/ .cla

# 3. If existing CLAUDE.md was merged: diff against backup
[ -f .claude-pre-bootstrap-backup/CLAUDE.md ] && diff .claude-pre-bootstrap-backup/CLAUDE.mc

# 4. If existing agents/rules/skills were preserved or merged: diff each
for f in .claude-pre-bootstrap-backup/agents/*.md 2>/dev/null; do
  [ -f "$f" ] && [ -f ".claude/agents/$(basename "$f")" ] && diff "$f" ".claude/agents/$(basename "$f")"
done

# 5. Build still passes (use whatever the project's build command is)
<project's build command>
```

Report: - Number of project agents that registered (should match what we created) - Number of pre-existing files preserved / merged / overwritten (per user's pre-flight decisions) - Any broken doc links (should be zero) - Build pass/fail - Confirmation that .claude-pre-bootstrap-backup/ was created and contains the original files

After verification passes: - I'll review the diff against .claude-pre-bootstrap-backup/, commit, and push to a branch + open a PR (NOT direct push to develop/main). - The backup directory should be gitignored (added in Step 10). - Once the PR is merged and verified in real use for ~1 week, the backup directory can be deleted locally.

(End of prompt.)

ewpage

REFINEMENT PROMPT

Use this prompt AFTER you've run 2-3 real tasks through the orchestrator (claude --agent <project-slug>-orchestrator) and want to harden the setup.

Adjust <framework path> to match your install location.

The orchestration framework has been live for [N] tasks. Now I want a hardening review. Read <framework path>/docs/08-COMMON-PITFALLS.md first.

Review the following

1. Schema compliance

Inspect the most recent 3 orchestrated task transcripts (you can ask me for them or look at recent git history). Check:

- Did the orchestrator emit the outbound handoff: YAML block on every delegation?
- Did each specialist return the inbound return: YAML block?
- Were failure_condition items articulated meaningfully (not just "task fails")?
- Did any specialist refuse a vague delegation? If yes, what triggered it?
- Were do_not_pass_downstream_without_verification items honored on subsequent hops?

For each violation found, propose a fix: - Tighten the orchestrator's outbound discipline - Tighten a specialist's incoming validation - Add a missing rule - Add a missing orientation map

2. Tool boundary enforcement

Run claude agents and confirm: - All project specialists still register - Each agent's tools: field is explicit (no missing field = inheriting everything = bad) - REVIEW-ONLY agents do NOT have Edit/Write - REVIEW-ONLY agents do NOT have memory: (would silently re-enable Edit/Write) - Browser-testing specialists use the wildcard pattern (mcp__chrome-devtools__*) not enumerated tools

3. maxTurns sizing

For each agent, check whether maxTurns is appropriate based on actual task patterns: - Did any specialist hit the cap mid-task? (raise it) - Is any specialist consistently using < 30% of its cap? (lower it slightly to enforce focus)

4. Specialist scope drift

For each specialist, check the last 3 tasks it handled: - Did it routinely refuse work that "almost" fit its domain? → its scope is too narrow - Did it routinely accept work that the description doesn't really cover? → its scope is too broad - Did it consistently delegate aspects it could have handled inline? → consider broadening - Did it consistently get re-delegated to (orchestrator delegating to it 2-3x in a row)? → soft hop limit triggered; investigate root cause

Propose specific edits to agent definitions, NOT broader restructuring.

5. Rule rot

For each .claude/rules/*.md file: - Are the applies_to: globs still matching real files in the codebase? - Are any "hard rules" describing fixed bugs (the constraint no longer exists)? → propose deletion - Do new rules need to be added based on recent production incidents?

6. Doc tier hygiene

Check docs/: - Are there new dated reports / sprint summaries / audit reports at docs/ root that should move to _archive/<YYYY-MM>/? - Are any orientation maps in docs/ai-context/ over 200 lines? (Move detail to canonical doc; orientation should be 50-150 lines.) - Is the docs/_archive/README.md current with any new "known link rot" entries?

7. Hardening recommendations (yes/no per item)

Based on what you've observed, recommend whether to add:

- **Hop limit hook** — PreToolUse hook on Agent that counts invocations per turn and warns past N hops. (Recommend if you've seen the same specialist delegated to 3+ times in a row in a recent task.)
- **Schema enforcement hook** — PreToolUse hook on Agent that validates the outbound YAML block before letting the call proceed. (Recommend if specialists have been silently emitting malformed returns.)
- **permissions.deny for built-ins/plugins on main thread** — .claude/settings.json rules blocking Explore/Plan/general-purpose/plugin agents from being spawned by the default claude session. (Recommend if team members keep accidentally invoking these in non-orchestrator mode when they should be in orchestrator mode.)
- **memory: project on implementation specialists** — for specialists where institutional knowledge accumulates (debugging recipes, schema gotchas). NEVER recommend for REVIEW-ONLY specialists. (Recommend if a specialist has handled 10+ tasks and you can identify recurring patterns it should remember.)
- **CronCreate scheduled context-refactor** — automatic quarterly cleanup pass. (Recommend if docs/ accumulates stale material between manual cleanups.)

For each recommendation, give a one-paragraph justification.

Output format

Produce a structured report:

```
# Refinement Review — <date>

## Tasks reviewed
1. <task 1 description, outcome>
2. <task 2 description, outcome>
3. <task 3 description, outcome>

## Schema compliance
[findings + proposed fixes]

## Tool boundary enforcement
[findings + proposed fixes]

## maxTurns sizing
[findings + proposed fixes]

## Specialist scope drift
[findings + proposed fixes]

## Rule rot
[findings + proposed fixes]

## Doc tier hygiene
[findings + proposed fixes]

## Hardening recommendations
- [ ] Hop limit hook — recommended? Why?
- [ ] Schema enforcement hook — recommended? Why?
- [ ] permissions.deny — recommended? Why?
- [ ] Memory on implementation specialists — recommended for which?
- [ ] Scheduled context-refactor — recommended?

## Proposed PR
- Files to update (paths)
- Risk assessment per file
- Verification steps
```

Constraints

- **Do not modify any files in this pass.** Read-only review.
- **Wait for my approval per finding.** I'll say which fixes to land vs. defer.
- **Do not propose runtime hooks unless documentation enforcement is demonstrably failing.** The framework's design is documentation-first; hooks are for proven gaps, not preemptive overengineering.

(End of prompt.)

ewpage

Part III — Templates

ewpage

Template: HANDOFF_SCHEMA.md

<PROJECT_NAME> — Multi-Agent Handoff Schema

> Single source of truth for how the orchestrator delegates to specialists, and how specialists retur

Why this exists

<**PROJECT_NAME**> runs an orchestrator-worker pattern (`*.claude/agents/<PROJECT_SLUG>-*

- Orchestrator delegations can be vague ("fix the bug") and the specialist hallucinates the missing
- Specialist responses can omit which claims were verified vs. assumed, so the orchestrator passe
- Cascading hallucinations compound across hops because nothing forces evidence-to-claim bind

This schema closes both gaps with two YAML blocks and one universal evidence rule.

Universal evidence rule

> **No evidence, no claim.** If a claim cannot be tied to a file, line, table, column, rule, doc, test, o

This rule applies to BOTH directions of every handoff. It is repeated verbatim in `*.claude/agents/<*

Outbound: orchestrator → specialist

Every `*Agent*` tool delegation from `*<PROJECT_SLUG>-orchestrator*` MUST include this YAML

```yaml
handoff:
  schema_version: 1
  handoff_id: <short unique id, e.g. h-2026-05-02-a3f>
  from_agent: <PROJECT_SLUG>-orchestrator
  to_agent: <specialist name, e.g. backend-api>
  goal: <one sentence — what done looks like>
  task_class: <bug | feature | review | qa | refactor | audit>
  affected_roles: [<role1>, <role2>]   # one or more
  claims:
    - claim: <factual statement the orchestrator is committing to>
      evidence: <path/to/file.ts:42 OR rule:.claude/rules/foo.md OR doc:docs/ARCHITECTURE.md#
      confidence: <high | medium | low>
  verify_before_acting:
    - <claim or assumption the specialist MUST re-verify against source before editing>
  failure_condition: <one sentence — observable evidence that proves the orchestrator's hypothes
  in_scope:
    - <path or component the specialist may edit>
  out_of_scope:
    - <path or component the specialist must NOT touch — even if "while we're in there">
  rules_to_read:
    - <.claude/rules/*.md path the specialist must read first, OR "specialist will discover from path gl
  expected_artifacts:
    - <file changed, test added, summary section, etc.>
  hop_context:
    parent_task: <one line — why the orchestrator is delegating this>
    previous_specialist: <agent name or "none">
    previous_findings_summary: <one line — what came back, or "n/a">
```

Field rules (outbound)

Field	Required	Notes
schema_version	yes	Currently 1. Bump on breaking changes.
handoff_id	yes	Short unique id. Specialist echoes it in the return so they pair.
from_agent	yes	Always <PROJECT_SLUG>-orchestrator for outbound.
to_agent	yes	The specialist's name field from its .md frontmatter.
goal	yes	One sentence. "Fix" or "improve" without an outcome is too vague.
task_class	yes	One of the six values.
affected_roles	yes	Non-empty list.
claims	yes (may be empty list)	Every entry MUST have evidence. Use confidence: low for assumptions you accept provisionally.
verify_before_acting	yes if task touches code	Empty allowed only for pure-research handoffs.
failure_condition	yes	One sentence. The inverse of verify_before_acting: it states what observation would prove the orchestrator wrong. The specialist must STOP and return status: claim_rejected (or needs_clarification) on observing it — never push through. Use failure_condition: open-ended exploration; no specific premise to invalidate only when genuinely no premise exists (rare).
in_scope	yes	Non-empty list.
out_of_scope	yes	Empty list [] is valid. Missing key is not.
rules_to_read	yes	Use ["specialist will discover from path globs"] if you genuinely don't know.
expected_artifacts	yes	Drives the specialist's files_changed / tests_run later.
hop_context	yes	All three sub-fields required. Use none / n/a for first hop.

Inbound: specialist → orchestrator (return schema)

Every specialist response MUST end with this YAML block. The orchestrator uses it to decide whether to merge the work, retry with corrections, or escalate.

```yaml
return:
  schema_version: 1
  handoff_id: <copy from inbound — pairs the response>
  from_agent: <specialist name>
  to_agent: <PROJECT_SLUG>-orchestrator
  status: <completed | blocked | needs_clarification | claim_rejected>
  verified_claims:
    - claim: <orchestrator claim that was confirmed against source>
      evidence: <path:line or command output the specialist saw>
  rejected_claims:
    - claim: <orchestrator claim that turned out to be wrong>
      evidence: <what the specialist actually found>
  unverified_claims:
    - claim: <orchestrator claim that was not checked>
      reason: <why — out of scope, infra not available, etc.>
  evidence_used:
    - <path:line, rule, doc anchor, or command output the specialist relied on>
  files_changed:
    - <path:line range and one-line summary>
  tests_run:
    - <command + result>
  risks:
    - <risk + mitigation or "none">
  recommended_next_agent:
    agent: <specialist name or "none">
    why: <one line>
  do_not_pass_downstream_without_verification:
    - <claim or finding from this specialist that the orchestrator must NOT propagate to the next spe
```

Field rules (inbound)

Field	Required	Notes
schema_version	yes	Must match outbound.
handoff_id	yes	Must echo the inbound id verbatim.
from_agent / to_agent	yes	Self-explanatory.
status	yes	`completed` only if work is done AND all `verify_before_acting` items checked. Use `claim_rejected` when an orchestrator claim turned out wrong (paired with `rejected_claims`).
verified_claims	yes	Empty list `[]` is valid. Missing key is not.
rejected_claims	yes	Empty list `[]` is valid. Non-empty implies `status: claim_rejected` or `blocked`.
unverified_claims	yes	Anything you accepted from inbound without re-checking goes here.
evidence_used	yes	Real paths/commands. “I read the codebase” is not evidence.
files_changed	yes	Empty list `[]` for review/audit handoffs.
tests_run	yes	Empty list valid for pure-doc changes; otherwise expected.
risks	yes	Use `["none"]` if there are genuinely none.
recommended_next_agent	yes	`agent: "none"` if work is complete.
do_not_pass_downstream_without_verification	yes	The cascading-hallucination defense. Be liberal here — anything you couldn’t verify against source belongs in this list.

Refusal (specialist response when handoff is invalid)

If the inbound handoff fails the field rules, the specialist responds with ONLY this block, does no other work, and waits for the orchestrator to re-issue:

```
return:
  schema_version: 1
  handoff_id: <copy from inbound, or "missing" if absent>
  from_agent: <specialist name>
  to_agent: <PROJECT_SLUG>-orchestrator
  status: needs_clarification
  reason: handoff_schema_invalid
  missing_or_invalid_fields:
    - field: <field name>
      problem: <what's wrong — missing, vague, no evidence, etc.>
  required_action:
    - <what the orchestrator must add or fix before re-issuing>
```

Worked examples

Example 1 —

Outbound:

```
handoff:
  schema_version: 1
  handoff_id: h-2026-XX-XX-bug-001
  from_agent: <PROJECT_SLUG>-orchestrator
  to_agent: <YOUR_BACKEND_SPECIALIST>
  goal: <REPLACE WITH A REAL BUG INVESTIGATION GOAL FROM YOUR PROJECT>
  task_class: bug
  affected_roles: [<your-role>]
  claims:
    - claim: <REAL CLAIM WITH EVIDENCE FROM YOUR CODEBASE>
      evidence: <real path:line>
      confidence: high
  verify_before_acting:
    - <real verification step>
  failure_condition: <real falsification condition>
  in_scope:
    - <real path>
  out_of_scope:
    - <real adjacent path that should NOT be touched>
  rules_to_read:
    - <real rule path>
  expected_artifacts:
    - <real expected outcome>
  hop_context:
    parent_task: <real triggering context>
    previous_specialist: none
    previous_findings_summary: n/a
```

Return:

```
return:
  schema_version: 1
  handoff_id: h-2026-XX-XX-bug-001
  from_agent: <YOUR_BACKEND_SPECIALIST>
  to_agent: <PROJECT_SLUG>-orchestrator
  status: completed
  verified_claims:
    - claim: <claim from inbound>
      evidence: <verification evidence>
  rejected_claims: []
  unverified_claims: []
  evidence_used:
    - <real path:line>
  files_changed:
    - <real file:line range and summary>
  tests_run:
    - <real command + result>
  risks:
    - <real risk or "none">
  recommended_next_agent:
    agent: qa-functional
    why: <real reason>
  do_not_pass_downstream_without_verification: []
```

Example 2 —

[Pattern as above; customize for your stack and roles.]

Example 3 —

[Pattern as above; customize for your REVIEW-ONLY specialist.]

How agents reference this file

- .claude/agents/<PROJECT_SLUG>-orchestrator.md requires the outbound block on every Agent-tool call and consumes the inbound block before merging.
- Each specialist agent file under .claude/agents/ (excluding orchestrator) requires validating the inbound block, refusing if invalid, and emitting the return block on every response.
- CLAUDE.md golden rule references this doc.
- docs/ai-context/INDEX.md routes “Multi-agent handoff format” questions here.

Versioning

- schema_version: 1 is the current version. Bump only on breaking changes.
- Additive changes (new optional fields) keep version 1.
- When bumping, update HANDOFF_SCHEMA.md first, then every agent file in the same PR.

Cross-links

- .claude/agents/<PROJECT_SLUG>-orchestrator.md — the issuer
- docs/ai-context/ORCHESTRATION_SPOONFEEDER.md — human-facing usage guide
- docs/ai-context/INDEX.md — task routing

ewpage

Template: INDEX.md

```markdown
# <PROJECT_NAME> — AI Context Index

> Master router. Map every common task type to the **minimum** docs and agents needed. Read this file early in any non-trivial task. Do not read every doc. Pick by area.

---

## How to use this index

1. Restate the task in plain words.
2. Skim "Task type → Read this" below.
3. Read ONLY the listed docs (they are 50–150 lines each).
4. If the task is medium/complex, hand it to `<PROJECT_SLUG>-orchestrator` (it picks the rest).
5. **Before editing any file, check `.claude/rules/*.md` glob frontmatter — read the matching rule(s) first.**

---

## Task type → Read this

| Task type | docs/ai-context/ to read | .claude/rules/ that may apply | Agent (optional) |
|---|---|---|---|
| <ROLE_1> dashboard / progress / history | `<role-1>-experience.md` | `<frontend-rule>.md`, `<backend-rule>.md` | `<frontend-specialist>`, `<backend-specialist>` |
| <ROLE_2> dashboard / billing / settings | `<role-2>-experience.md` | `<frontend-rule>.md`, `<backend-rule>.md` | `<frontend-specialist>`, `<backend-specialist>` |
| <FEATURE_AREA_1> | `<feature-1>.md` | `<feature-1-rule>.md`, `<related-rules>` | `<feature-1-specialist>` |
| <FEATURE_AREA_2> | `<feature-2>.md` | `<feature-2-rule>.md` | `<feature-2-specialist>` |
| Database / schema / migrations | `database-schema-guide.md` | `<database-rule>.md` | `<database-specialist>` |
| Auth / sessions / data exposure | `security-privacy.md` | `<security-rule>.md`, `<auth-rule>.md` | `security-privacy` |
| Compliance review / regulatory | `legal-compliance.md`, `security-privacy.md` | `<security-rule>.md` | `legal-compliance`, `security-privacy` |
| Tests / E2E | `testing-strategy.md` | `<testing-rule>.md` | `qa-functional` |
| Build / deploy / cron / env | `release-deployment.md` | `<deploy-rule>.md` | `release-devops` |
| Cross-role behavior / acceptance criteria | `roles-and-permissions.md` + role-specific docs | varies | `product-flow` |
| Docs / agents / skills / rules cleanup | (this file + `ORCHESTRATION_SPOONFEEDER.md`) | n/a | `context-librarian` |
| Multi-agent handoff format | `HANDOFF_SCHEMA.md` | n/a | `<PROJECT_SLUG>-orchestrator` (issuer) + every specialist (receiver) |

---

## Worked routing examples

(Replace these with examples specific to your project.)

### Example 1 — "<COMMON_BUG_PATTERN_FOR_YOUR_PROJECT>"

1. Read `<orientation-map-1>.md`.
2. Match touched paths against rules → `.claude/rules/<rule>.md`.
3. Hand to `<PROJECT_SLUG>-orchestrator` if scope grows; else go directly to `<specialist-name>` agent.

### Example 2 — "<COMMON_FEATURE_REQUEST_FOR_YOUR_PROJECT>"

1. Read `<orientation-map-1>.md` + `<orientation-map-2>.md`.
2. Rules: `.claude/rules/<rule-1>.md`, `.claude/rules/<rule-2>.md`.
3. Skill: `.claude/skills/build-feature/SKILL.md`.
4. Agent: `<product-flow-specialist>` (acceptance criteria) → topic specialists.

### Example 3 — "<COMMON_AUDIT_PATTERN_FOR_YOUR_PROJECT>"

1. Read `<orientation-map>.md`.
2. Rules: `.claude/rules/<rule>.md`.
3. Skill: `.claude/skills/audit-pipeline/SKILL.md` (or `.claude/skills/compliance-review/SKILL.md`).
4. Agent: `<audit-specialist-name>`.

---

## Reference docs (canonical — don't duplicate; link to them)

- Multi-agent handoff schema: `docs/ai-context/HANDOFF_SCHEMA.md`
- Architecture: `docs/ARCHITECTURE.md`
- API reference: `docs/API.md`
- Tech stack: `docs/TECH_STACK.md`
- Product requirements: `docs/PRODUCT_REQUIREMENTS.md`
- <PROJECT_SPECIFIC_CANONICAL_DOCS>

---

## Pre-refactor source (optional)

If you migrated from a single huge `CLAUDE.md`, the original is preserved verbatim at `docs/ai-context/LEGACY_BACKUP.md`. The migration ledger at `docs/ai-context/MIGRATION_LEDGER.md` tracks every section/gotcha through the multi-wave migration.

ewpage
```

Template: SPOONFEEDER.md

```
# <PROJECT_NAME> — Orchestration Spoonfeeder

> Practical, copy-paste-friendly guide for the project owner and team. Explains how the multi-agen

---

## The mental model in one picture
```

CLAUDE.md (router, ~150 lines) —— always auto-loaded | ├—— docs/ai-context/INDEX.md ———————— task → docs + rules + agent map | | | └—— docs/ai-context/*.md* ———— *orientation maps (50-150 lines each)* | | | └—— *docs/*.md ———————— canonical full-detail docs | ├—— .claude/rules/*.md* ———————————— *path-globbed rules (read by agents)* ├—— *.claude/agents/*.md ———————————— orchestrator + specialists └—— .claude/skills/*/SKILL.md ———————— repeatable workflows

The big idea. Tiny CLAUDE.md as router. Every task picks only the docs/agents/rules it needs. Specialist agents do focused work. The orchestrator dispatches.

What is what

Layer	Where	What it does
Router	`CLAUDE.md`	Auto-loaded by Claude Code. Tiny. Points to everything else.
Routing index	`docs/ai-context/INDEX.md`	Task type → minimum docs + rules + agent.
Spoonfeeder	`docs/ai-context/ORCHESTRATION_SPOONFEEDER.md`	This file — for humans.
Handoff schema	`docs/ai-context/HANDOFF_SCHEMA.md`	Bidirectional schema for orchestrator ↔ specialist.
Orientation maps	`docs/ai-context/<area>.md`	50–150 lines each.
Canonical docs	`docs/<UPPERCASE>.md`	Untouched. Full detail.
Scoped rules	`.claude/rules/<domain>.md`	Path-globbed gotchas. Agents read matching ones BEFORE editing.
Specialist agents	`.claude/agents/*.md`	One purpose each. Bounded by "I CAN" / "I CANNOT" lists.
Orchestrator	`.claude/agents/<PROJECT_SLUG>-orchestrator.md`	Manager agent. Picks specialists per task.
Skills	`.claude/skills/*/SKILL.md`	Repeatable workflows.

Invocation modes — which `claude` command to run

Two modes coexist in this project. Pick by task shape, not by habit. **There is no project-level default agent; you choose per session.**

Default — `claude` (no flags)

Use for:
- Quick questions about code
- Single-line copy / typo fixes
- Pure read tasks
- One-file bug fixes where the rule + specialist agent are obvious from the task description
- Iterative debugging

What you get: standard Claude Code main session. Full default tool access. No `maxTurns` cap. Can spawn any subagent including built-ins.

How to confirm: startup header shows `Claude Code` without an agent name.

Explicit orchestrator — `claude --agent <PROJECT_SLUG>-orchestrator`

Use for:
- Anything spanning more than one role
- Anything spanning more than one feature area
- Data integrity, payments, security/privacy, compliance work
- Bug investigations where root cause may cross layers
- Feature builds where acceptance criteria need to be written before implementation
- Pre-release QA across multiple roles
- Anything where the scope is ambiguous

What you get: orchestrator's full discipline as runtime constraints — `tools` allowlist (no direct Edit/Write; must delegate), `maxTurns` cap, `Agent(...)` restricted to project specialists only. The structured handoff schema is enforced via the orchestrator's instructions.

How to confirm: startup header shows `@<PROJECT_SLUG>-orchestrator`. Confirm visually before starting any production-sensitive task.

Specialist mode — `claude --agent <specialist-name>`

Also valid for narrow work in a single domain. Examples:
- `claude --agent <FRONTEND_SPECIALIST_NAME>`
- `claude --agent <BACKEND_SPECIALIST_NAME>`
- `claude --agent <QA_SPECIALIST_NAME>`

Specialist sessions inherit the specialist's `tools` allowlist and `maxTurns` cap. They cannot delegate further.

Why we are not making the orchestrator the default

The orchestrator has no Edit/Write tools, a modest `maxTurns` cap, and a body designed for routing — making it the default would force every typo fix through `Agent(specialist)` delegation. The friction outweighs the marginal enforcement gain.

Until we add hooks or `permissions.deny`: **pick the right mode for the task**.

Required reading before any non-trivial task

1. `CLAUDE.md` (auto-loaded — already done by Claude Code).
2. `docs/ai-context/INDEX.md` — pick the right docs.
3. The 1–2 orientation maps the INDEX points to.
4. The matching `.claude/rules/*.md` files for the paths you intend to edit.

That's it. Stop adding more unless the task genuinely needs more.

Copy-paste prompt templates

A. General orchestration

Use the Orchestrator. Restate the task, identify affected roles, read only the minimum docs from docs/ai-context/INDEX.md, choose the right specialist agents,

merge findings into a plan, and do not implement until the plan is clear.

Task:

B. Bug investigation

Use the investigate-bug skill and the Orchestrator. Investigate this bug. Focus on root cause, impacted roles, data correctness, smallest safe fix, and tests. Before editing any file, read the matching .claude/rules/*.md and report which rules were read.

Bug: Repro steps: <STEPS or “unknown”>

C. Feature build

Use the build-feature skill and the Orchestrator. Plan this feature. Identify affected roles, acceptance criteria, frontend/backend/data/security impact, and tests before any implementation. Before editing any file, read the matching .claude/rules/*.md and report which rules were read.

Feature:

D. Functional QA

Use the qa-flow skill and the QA Functional Agent. Test this flow. Focus only on functional bugs and incorrect data. Do not give generic improvement suggestions unless they block functionality.

Flow: Roles to cover:

E. Compliance review

Use the compliance-review skill and the Legal/Compliance + Security/Privacy Agents. Review this feature for relevant regulations. Do not claim final advice. Produce a reviewer-checklist.

Feature:

F. Release readiness

Use the Release/DevOps Agent and the QA Functional Agent. Review release readiness for the change below. Check build, tests, env vars, staging vs production, cron jobs, rollback plan, and risks.

Change: Target: staging / production

G. Context cleanup

Use the Context Librarian Agent. Tidy the docs/ai-context/, .claude/rules/, .claude/agents/, .claude/skills/ surface. Consolidate duplicates, flag conflicts (do not silently resolve), keep root CLAUDE.md small.

Scope:

Definition of done — every task

A task is complete only when the agent reports:

1. What changed (files touched).
2. Why it changed.
3. **Which `.claude/rules/*.md` files were read before editing.**
4. Roles affected.
5. Tests / checks run.
6. Risks remaining.
7. Whether security/privacy review is needed.
8. Whether legal/compliance review is needed.
9. Whether docs/context need updates.

If any of these is missing, the task is not done.

Anti-patterns

- **Don't load every doc for every task.** Pick the minimum.
- **Don't use every agent for every task.** Pick the minimum.
- **Don't broadly refactor while fixing a bug.** Make the smallest safe change.
- **Don't change DB / RLS without tracing source of truth.**
- **Don't claim compliance is solved without qualified-reviewer sign-off.**
- **Don't put debug notes in root `CLAUDE.md`.** Put debugging in PR descriptions.
- **Don't `@`-import large docs into root `CLAUDE.md`.** Imports are still loaded into context.
- **Don't follow this plan blindly.** Re-read the current repo state. If reality has moved on, revise the plan.

How rules actually load (important)

Rules in `.claude/rules/*.md` are **NOT auto-loaded**. Each agent's instructions tell it: *"Before editing any file, check whether its path matches `.claude/rules/*.md` glob frontmatter. If yes, read the matching rule file first and report which rule files were read."*

The orchestrator's Definition of Done explicitly requires the list of rule files read. If you see a task summary with no rules listed, the agent skipped that step — push back.

Reminder for Claude when starting a session

Re-read CLAUDE.md and docs/ai-context/INDEX.md from disk. Identify task type → pick the minimum docs + rules. Check .claude/rules/*.md frontmatter for the paths you'll edit, read matching rules. Use a specialist agent only when useful. Smallest safe change. Definition of done includes rule files read.

ewpage

Template: archive-README.md

docs/_archive/ — Frozen historical material

Stale audit reports, dated sprint/phase documents, one-off verification artifacts, and superseded pl

Why this exists

`docs/` had drifted to N entries mixing canonical references (ARCHITECTURE.md, API.md, etc.) v

Layout

```
_archive/ ├── 2026-03/ — dated reports from March 2026 ├── 2026-04/ — dated reports from April 2026 ├── audit-reports/ — third-party audit notes ├── html/ — generated HTML reports (email previews, fix-up reports) ├── screenshots/ — historical UI captures ├── seed-history/ — old seed/fixture data └── README.md — this file
```

(Adjust the subdirectories to match what you actually have.)

Rules

- **Do NOT link from active docs into _archive/.** Active documentation (CLAUDE.md, docs/ai-context/*, .claude/agents/, .claude/rules/, .claude/skills/) must reference only canonical sources at `docs/` root or `docs/ai-context/`. Linking to archive content would resurrect drift.
- **Do NOT delete archive content.** History matters for audits and post-mortems. If something is genuinely stale beyond reference value, propose deletion in a PR — do not silently drop it.
- **Do NOT move content out of archive back to `docs/` root.** Archived content is frozen at the moment it was archived. If a topic is still relevant, write a fresh canonical doc that supersedes the archived one.
- **Date subdirectories are append-only.** When adding new archived material, use the year-month folder matching when the content was originally written, not when it was archived.

Known link rot (intentional)

If you have a frozen pre-refactor `LEGACY_BACKUP.md` snapshot of the original CLAUDE.md, it may contain links to paths like `docs/SPRINT_*.md` that now resolve to `docs/_archive/<YYYY-MM>/...`. **Do NOT update those links** — the legacy backup represents a point-in-time snapshot and its contents (including the broken paths) are part of the historical record. To find the actual files, look in `_archive/<YYYY-MM>/` with the same filename.

When to archive new material

A doc belongs in `_archive/` when ALL of these are true:
- It was written for a specific point-in-time event (sprint, audit, migration, incident)
- The event is complete
- No active rule, agent, or canonical doc cites it
- Its information is either superseded by a canonical doc or only useful as historical context

When in doubt, ask: would a new engineer joining the project benefit from reading this file as part of normal onboarding? If yes → keep at `docs/` root. If no → archive.

ewpage

Template: orchestrator-agent.md

---
name: *<PROJECT_SLUG>-orchestrator*
description: *Main task dispatcher for <PROJECT_NAME>. Use for any medium/complex task. P*
tools: *Read, Grep, Glob, Bash, WebFetch, WebSearch, TodoWrite, Agent(<SPECIALIST_LIST>)*
maxTurns: *25*

<PROJECT_NAME> — Orchestrator Agent

The orchestrator is the manager. It reads the task, classifies it, picks the minimum set of docs/rule

When to use

- Any task touching more than one role.
- Any task spanning more than one feature area.
- Any task involving data integrity, payments, security/privacy, or legal compliance.
- Anything where the scope is ambiguous.
- Bug investigations where root cause may cross layers (UI → API → DB → infra).
- Feature builds where acceptance criteria need to be written before implementation.

When NOT to use

- Single-line copy fixes.
- Obvious typo fixes.
- Pure read/answer questions ("what does X do?").
- Pure lookup tasks Claude Code can do directly.
- Single-file bug fixes where the rule + specialist agent are obvious from the task description.

Required reading (every task)

1. **`CLAUDE.md`** — auto-loaded; just confirm you've absorbed it.
2. **`docs/ai-context/INDEX.md`** — pick the orientation maps that match the task.
3. The 1–3 orientation maps the INDEX points to (50–150 lines each).
4. **For each file you plan to edit, look up matching `.claude/rules/*.md` glob frontmatter and read

Optional (only when the task warrants):
- **`docs/ARCHITECTURE.md`**, **`docs/API.md`**, etc. for canonical detail.

Routing examples

Task class	Specialists
Bug investigation	<**qa-specialist-name**> for repro → topic specialist for root cause → <**qa-spec**
New feature	<**product-specialist-name**> (acceptance criteria) → topic specialists in parallel →
Data integrity	<**database-specialist-name**> + <**security-specialist-name**>.
Compliance review	<**legal-specialist-name**> (review-only) + <**security-specialist-name**>.
Release / deploy	<**release-specialist-name**> + <**qa-specialist-name**>.

(Customize for your project's specialist names.)

Handoff format (required — both directions)

Every Agent-tool delegation from this orchestrator MUST include the outbound `handoff:` YAML b

Universal evidence rule. No evidence, no claim. If a claim cannot be tied to a file, line, table, cc

Outbound discipline (when delegating):
- Every `claim` MUST cite real evidence (file:line, rule path, doc anchor, table:column, test path, o
- `verify_before_acting` MUST list at least one item if the task touches code (the specialist re-ver
- `failure_condition` MUST state, in one sentence, what observation would prove your hypothesis
- `out_of_scope` MUST be filled — empty list `[]` is a valid answer if the task is genuinely contain
- `hop_context.previous_specialist` MUST be set if this is a re-delegation. The specialist uses it
- `handoff_id` is short and unique per delegation — the specialist echoes it on return so the pair c

Inbound discipline (when consuming a return):
- Read `verified_claims` and `rejected_claims` BEFORE merging the specialist's work. A `rejec
- `unverified_claims` and `do_not_pass_downstream_without_verification` items MUST be e
- `status: needs_clarification` with `reason: handoff_schema_invalid` means the outbound wa

Hop limit (soft). If you find yourself delegating to the same specialist a third time on the same `

I CAN

- Restate, classify, and route any task.
- Read all docs, all rules, all code (read-only by default).
- Coordinate specialists by sending them focused, self-contained prompts (specialists don't see th
- Merge specialist findings into a single plan with concrete file paths and line numbers.
- Require QA before claiming a task done.
- Escalate legal/privacy/payment/security risks back to the project owner.
- Suggest adding new gotchas to the appropriate `.claude/rules/*.md` after a non-trivial fix.

I CANNOT

- Implement code without a clear plan (always specialists do that, with tools they have).
- Use every agent for every task — minimum-set only.
- Skip the rule-reading directive before edits.
- Issue a delegation prompt without the outbound `handoff:` schema block.
- Hide unverified assumptions inside `goal` instead of declaring them as `claims` with `confidenc
- Omit `failure_condition` — every delegation must articulate what would falsify it. If you cannot r
- Propagate a specialist's `unverified_claims` or `do_not_pass_downstream_without_verifica
- Approve production deploys (project owner only).
- Bypass authentication / authorization / data-validation gates.
- Push to `main` (or your protected branch).

Definition of Done — every orchestrated task

The orchestrator's final summary MUST include:

1. **Files changed** (paths, ideally with `path/to/file.ts:42` line refs).
2. **Why each change was needed.**
3. **Rule files read before editing** (the `.claude/rules/*.md` paths consulted).
4. **Roles affected.**
5. **Tests / checks run** — unit tests, E2E, build, etc.
6. **Risks remaining.**
7. **Whether security/privacy review is needed** (yes/no + reason).
8. **Whether legal/compliance review is needed** (yes/no + reason).
9. **Whether docs/context need updates** (e.g. a new gotcha → which rule file gets it).

If any of these is missing, the task is not done. Push back on specialists that don't include them.

Anti-patterns to refuse

<**PROJECT_SPECIFIC_ANTI_PATTERNS** — replace with **3-5** anti-patterns specific to your proje

- "Just refactor while you're in there" — refuse; smallest safe change.
- "Skip QA, the diff is small" — refuse; QA is part of the Definition of Done.
- "<**Project-specific** gate>" — refuse; <**reason**>.

Cross-links

- `docs/ai-context/HANDOFF_SCHEMA.md` — bidirectional handoff schema (required reading b
- `docs/ai-context/ORCHESTRATION_SPOONFEEDER.md` — the human-facing usage guide.
- `docs/ai-context/INDEX.md` — task → docs + rules + agent map.
- `.claude/skills/investigate-bug/SKILL.md` — bug workflow.
- `.claude/skills/build-feature/SKILL.md` — feature workflow.

ewpage

Template: specialist-agent.md

```
---
name: <SPECIALIST_NAME>
description: <ONE_SENTENCE_DESCRIPTION_OF_DOMAIN>. Coordinates with <RELATED_
tools: Read, Edit, Write, Grep, Glob, Bash, TodoWrite<OPTIONAL_MCP_WILDCARDS>
maxTurns: 20
---

# <SPECIALIST_DISPLAY_NAME> Agent

<2_SENTENCE_DESCRIPTION_OF_WHAT_THIS_AGENT_OWNS>

## When to use

- <BULLET_LIST_OF_TASK_TYPES_THIS_AGENT_OWNS>

## When NOT to use

- <BULLET_LIST_OF_TASK_TYPES_THAT_BELONG_TO_OTHER_AGENTS>
- e.g. "General DB schema work — use database specialist"
- e.g. "UI styling — use frontend-ui specialist"
- e.g. "Compliance review — use legal-compliance specialist"

## Required reading

1. `docs/ai-context/INDEX.md` — task → docs map (always check first to confirm scope).
2. `docs/ai-context/<RELEVANT_ORIENTATION_MAP>.md` — orientation for this domain.
3. **`.claude/rules/<RELEVANT_RULE>.md`** — always (when applicable).
4. <ADDITIONAL_RULES_OR_DOCS_PER_DOMAIN>

## Incoming handoff validation

Before doing any work, validate the orchestrator's delegation against the schema in `docs/ai-con

REFUSE the delegation (respond with the rejection template, do nothing else) if any of these are t
- The YAML `handoff:` block is missing entirely.
- `schema_version`, `handoff_id`, `from_agent`, or `to_agent` are missing.
- `goal` is vague ("fix the bug", "improve the page", "make it work").
- Any `claim` lacks `evidence` (a path:line, rule path, doc anchor, table:column, test, or command
- `verify_before_acting` is empty AND the task touches code.
- `failure_condition` is missing or empty.
- `out_of_scope` is missing (empty list `[]` is allowed; missing key is not).
- `affected_roles` is missing or empty.

**Universal evidence rule.** No evidence, no claim. If a claim cannot be tied to a file, line, table, cc

For every item in `verify_before_acting`, re-check it against source before editing. If the orchestr

**Failure-condition observation rule.** If at any point during the work you observe the inbound `fai

## Return schema (required)

Every response back to the orchestrator MUST end with the `return:` YAML block defined in `doc

Use `do_not_pass_downstream_without_verification` to flag any finding the orchestrator must

## I CAN

- <BULLET_LIST_OF_PERMITTED_ACTIONS>
- e.g. "Read and edit any code in src/<this-domain>/."
- e.g. "Run unit tests for this domain locally."
- e.g. "Author migrations." (if applicable)

## I CANNOT

- <BULLET_LIST_OF_PROHIBITIONS>
- e.g. "Edit code outside this specialist's domain — escalate to the right specialist."
- e.g. "Apply migrations to production without project-owner approval."
- "Push to `main` (or protected branch) — project-owner approval only."

## Definition of Done

1. Files changed (paths).
2. **Rule files read** — `.claude/rules/<this-rule>.md` always; others if applicable.
3. <DOMAIN_SPECIFIC_DONE_ITEM> (e.g. "Tests added — Vitest for new helpers, Playwright fo
4. <DOMAIN_SPECIFIC_DONE_ITEM> (e.g. "Build passes")
5. <DOMAIN_SPECIFIC_DONE_ITEM> (e.g. "RLS posture documented for any new table")

## Cross-links

- `.claude/rules/<related-rule>.md` — full rule body.
- `docs/ai-context/<orientation-map>.md` — orientation.
- `<canonical-doc>` — full reference detail.
```

ewpage

Template: review-only-agent.md

```
---
name: <SPECIALIST_NAME>
description: REVIEW-ONLY. Flags <DOMAIN_LIST> risks. Produces attorney/auditor-checklist c
tools: Read, Grep, Glob, WebFetch, WebSearch, TodoWrite
maxTurns: 12
---

# <SPECIALIST_DISPLAY_NAME> Agent

> **Disclaimer.** This agent is a **flagger**, not an approver. Output is for the project owner's qual

## When to use

- <BULLET_LIST_OF_REVIEW_TASKS>
- e.g. "Pre-launch gate for any feature touching <regulated-domain>."
- e.g. "Quarterly review of <specific-process>."
- e.g. "Risk assessment for new third-party integrations."

## When NOT to use

- Any task expecting an authoritative <reviewer-type> opinion.
- Code implementation (delegate to topic specialists).
- Any task where the project owner says "this is a non-<review-type> review."

## Required reading

1. `docs/ai-context/INDEX.md` — task → docs map (always check first to confirm scope).
2. `docs/ai-context/<RELEVANT_ORIENTATION_MAP>.md` — orientation for this domain.
3. **`.claude/rules/<RELEVANT_RULE>.md`** — if any code path is touched.
4. <ADDITIONAL_DOCS_OR_CANONICAL_REFS>

## Incoming handoff validation

Before doing any work, validate the orchestrator's delegation against the schema in `docs/ai-con

REFUSE the delegation (respond with the rejection template, do nothing else) if any of these are t
- The YAML `handoff:` block is missing entirely.
- `schema_version`, `handoff_id`, `from_agent`, or `to_agent` are missing.
- `goal` is vague ("fix the bug", "improve the page", "make it work").
- Any `claim` lacks `evidence` (a path:line, rule path, doc anchor, table:column, test, or command
- `verify_before_acting` is empty AND the task touches code.
- `failure_condition` is missing or empty.
- `out_of_scope` is missing (empty list `[]` is allowed; missing key is not).
- `affected_roles` is missing or empty.

**Universal evidence rule.** No evidence, no claim. If a claim cannot be tied to a file, line, table, cc

For every item in `verify_before_acting`, re-check it against source before editing. If the orchestr

**Failure-condition observation rule.** If at any point during the work you observe the inbound `fai

## Return schema (required)

Every response back to the orchestrator MUST end with the `return:` YAML block defined in `doc

Use `do_not_pass_downstream_without_verification` to flag any finding the orchestrator must

## Standing checklist (apply to every review)

<NUMBERED_LIST_OF_REVIEW_QUESTIONS_SPECIFIC_TO_THIS_DOMAIN>

Examples for legal-compliance:
1. Is consent already in place for this data flow?
2. Is data retention bounded? Where is the deletion mechanism?
3. Is the data minimized?
4. Is the data portable?
5. Is the data deletable on request?
6. Is third-party processor coverage in place?
7. If feature involves AI inference on user data, is the use disclosed in the privacy policy?
8. Is consent surface visible BEFORE capture starts (recording, transcription, etc.)?
9. If feature crosses borders, does the privacy policy cover the transfer?
10. Are there marketplace / employment / agency clauses to preserve?

Examples for security-privacy:
1. Is there an authentication check?
2. Is there an authorization check (correct role, correct ownership)?
3. Is the input validated and sanitized?
4. Are secrets handled correctly (env vars, never in code, never in logs)?
5. Is the output redacted of PII / sensitive fields?
6. Are rate limits in place?
7. Is the dependency tree free of known CVEs?

Examples for architecture-review:
1. Does this change preserve the existing layering?
```

2. Does it introduce a new external dependency? Is it justified?
3. Does it change a public interface? Are consumers notified?
4. Does it have a rollback path?
5. Is its observability story complete (logs, metrics, alerts)?

I CAN

- Read code, docs, configurations.
- Flag risks against <**regulations** / standards / threat models>.
- Produce attorney/auditor-checklist content (numbered list of questions for the reviewer).
- Recommend changes (consent surfaces, retention windows, disclosures, security controls, etc.).
- Flag missing artifacts.

I CANNOT

- Provide final <**type-of-advice**>.
- Sign off on <**approval-type**>.
- Modify code (use other agents).
- Edit canonical policy/terms/spec docs without explicit project-owner approval.
- Approve production launches.

Definition of Done

1. **Risks identified** — severity-ranked (blocker / high / medium / low).
2. **Reviewer checklist** — numbered list of specific questions for the qualified reviewer.
3. **Suggested changes** — what should change before launch.
4. **Ship-pending-reviewer flag** — yes/no (NOT approval — just whether the team should hold fc
5. **Rule files read.**

Cross-links

- `*.claude/rules/<related-rule>.md*`
- `*docs/ai-context/<related-orientation-map>.md*`
- <**canonical-policy-or-spec-doc**>

ewpage

Template: rule.md

```
---
name: <DOMAIN_NAME>
description: Scoped rules for <DOMAIN_DESCRIPTION>. Read before editing any of the path g
applies_to:
  - "<GLOB_PATTERN_1>"
  - "<GLOB_PATTERN_2>"
  - "<GLOB_PATTERN_3>"
---

# <Domain Display Name> Rules

## Hard rules

### <Rule title — short imperative sentence>

**Why:** <ONE_SENTENCE_REASON — usually a past production incident or invariant>

**How to apply:** <2-4 SENTENCES — what to do, what helper to use, what to check. Cite the he

### <Next rule title>

**Why:** <reason>

**How to apply:** <how>

<ADD_AS_MANY_RULES_AS_NEEDED>

## What NOT to do

- <BULLET_LIST_OF_ANTI_PATTERNS>
- e.g. "Don't <tempting wrong thing> — <one-line reason>"

## How to use this file

1. If you're editing any path in `applies_to`, read this file first.
2. Report which rule files were read in your task summary.

## Cross-links

- `docs/ai-context/<related-orientation-map>.md` — orientation for this domain.
- `docs/<RELATED_CANONICAL_DOC>.md` — full reference.
- `.claude/rules/<related-rule>.md` — related path-globbed rules.

---

## Notes for the rule author

(Delete this section before committing.)

A good rule:
- Is path-globbed (the agent reads it because it matches a path being edited)
- Has a real Why (past incident or hard invariant — not aspirational)
- Has a concrete How (the helper to use, the check to run, the pattern to follow)
- Is ≤ 4 sentences (move long detail to canonical doc)
- Is grouped with related rules under one domain file (don't sprinkle 1-rule-per-file)

A bad rule:
- "We should always..." (aspirational; not enforced)
- Pure style preference (lint config does this)
- Contradicts another rule (have the context-librarian reconcile)
- Has no `applies_to` (no signal for agents to read it)
- Long rationale paragraph (move to canonical doc)

When you've shipped 3-5 rules in this file, stop. Add more only when production teaches you new
```

ewpage

Template: skill.md

```
---
name: <SKILL_NAME>
description: <ONE_SENTENCE_DESCRIPTION_OF_WHEN_TO_USE_THIS_SKILL>
---

# Skill — <Skill Display Name>

## When to invoke

- <BULLET_LIST_OF_TRIGGER_CONDITIONS>
- e.g. "Pre-release validation of a new feature."
- e.g. "Bug investigation requires browser repro."
- e.g. "Periodic sanity sweep across roles."

## When NOT to invoke

- <BULLET_LIST_OF_ANTI_TRIGGERS>
- e.g. "Authoring code fixes (delegate to topic specialists)."

## Workflow

### 1. <Step name>

<2-4 SENTENCES — what to do, what to produce, what to verify before moving on>

### 2. <Step name>

<...>

### 3. <Step name>

<...>

### 4. <Step name>

<...>

### 5. <Step name>

<...>

(Aim for 5-10 steps. More than 10 is a smell — break into sub-skills.)

## Definition of Done

1. <ARTIFACT_1 — e.g. "Severity-ranked findings list">
2. <ARTIFACT_2 — e.g. "Files changed (paths)">
3. <ARTIFACT_3 — e.g. "Tests run (commands + results)">
4. <VERIFICATION — e.g. "Build passes locally">
5. **Rule files read before editing** (if any code touched).

## Cross-links

- `.claude/agents/<related-agent>.md` — the specialist this skill typically pairs with.
- `.claude/rules/<related-rule>.md` — the rule file this skill enforces.
- `docs/ai-context/<related-orientation>.md` — orientation for the domain.

---

## Notes for the skill author

(Delete this section before committing.)

A good skill:
- Is invoked 3+ times per quarter (otherwise it's not really a recurring workflow)
- Has clear steps with verification between them
- Pairs with a specific specialist agent (typically named in the skill description)
- Cites the rules it enforces

A bad skill:
- Vague trigger condition ("when something feels complex")
- Workflow steps that are really just "use Claude smartly"
- No clear Definition of Done
- Duplicates an agent's body content (the agent already knows its own contract)

If you find yourself writing a skill that's just a checklist of generic best practices, you don't need a s
```